



Compute Accelerators (GPUs and more)

Chapter 10



Lecture overview

Parallel programming and why?

GPU vs CPU

Memory in GPU and its management

Overview of previous important concepts in a new context

- Amdahl law and pipelining

- SIMD, VLIW

- Cache

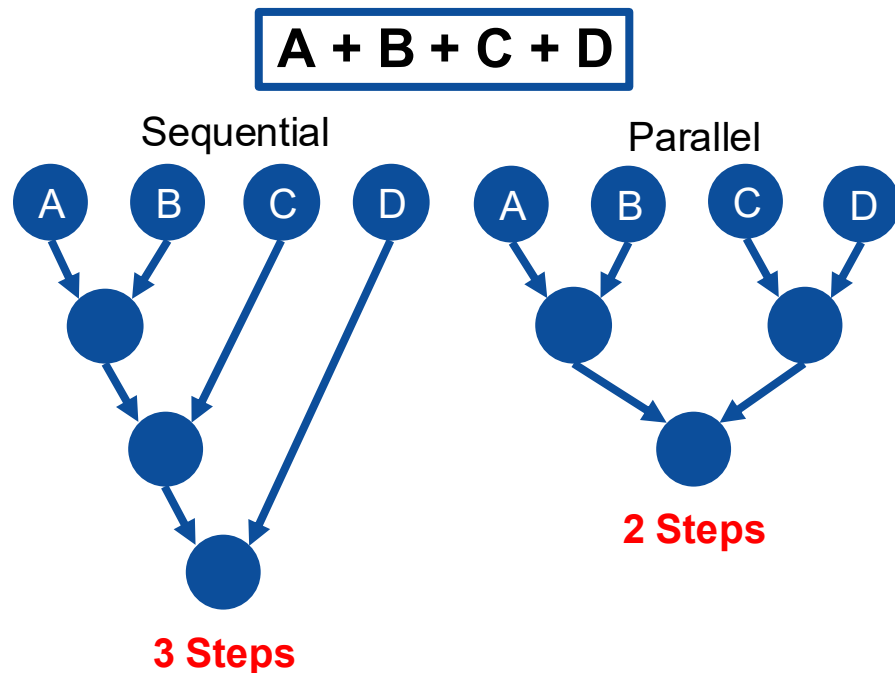
- Memory hierarchy

- VM

INTRODUCTION TO PARALLEL PROGRAMMING

WHAT IS PARALLEL PROGRAMMING?

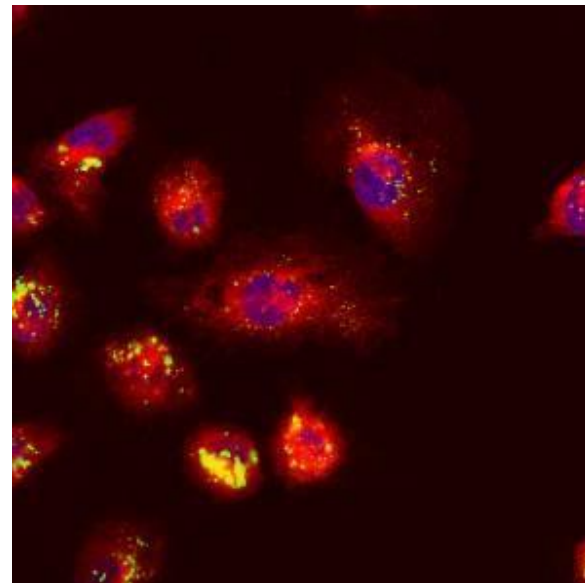
- **“Performance Programming”**
- Parallel programming involves exposing an algorithm’s ability to execute in parallel
- This may involve breaking a large operation into smaller tasks (task parallelism)
- Or doing the same operation on multiple data elements (data parallelism)
- Parallel execution enables better performance on modern hardware



A REAL WORLD CASE STUDY

Modern cancer research

- The Russian Academy of Science created a program to simulate light propagation through human tissue
- This program was used to be able to more accurately detect cancerous cells by simulating **billions** of random paths that the light could take through human tissue
- With parallel programming, they were able to run **thousands** of these paths **simultaneously**
- The sequential program took **2.5 hours** to run
- The parallel version took less than **2 minutes**



WHAT IS PARALLEL PROGRAMMING?

A real world example

- A professor and his 3 teaching assistants (TA) are grading 1,000 student exams
- This exam has 8 questions on it
- Let's assume it takes 1 minute to grade 1 question on 1 exam
- To maintain fairness, if someone grades a question (for example, question #1) then they must grade that question on all other exams
- The following is a sequential version of exam grading

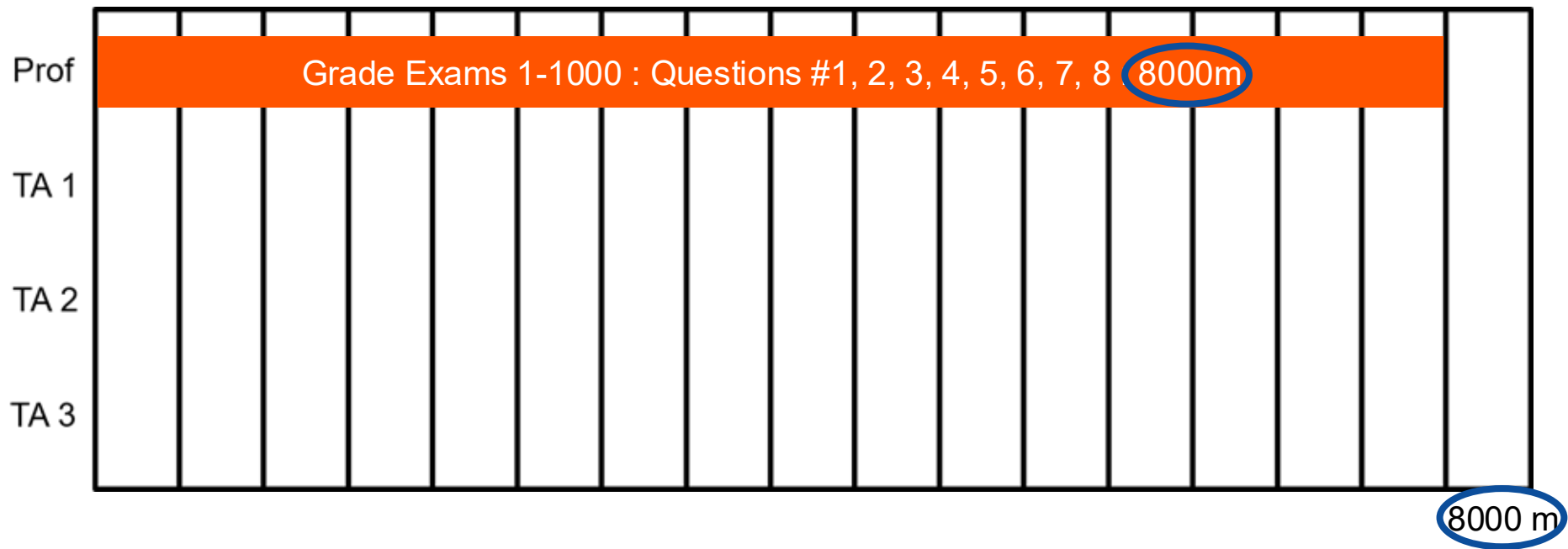


x1000
8 questions per exam
8,000 questions in total

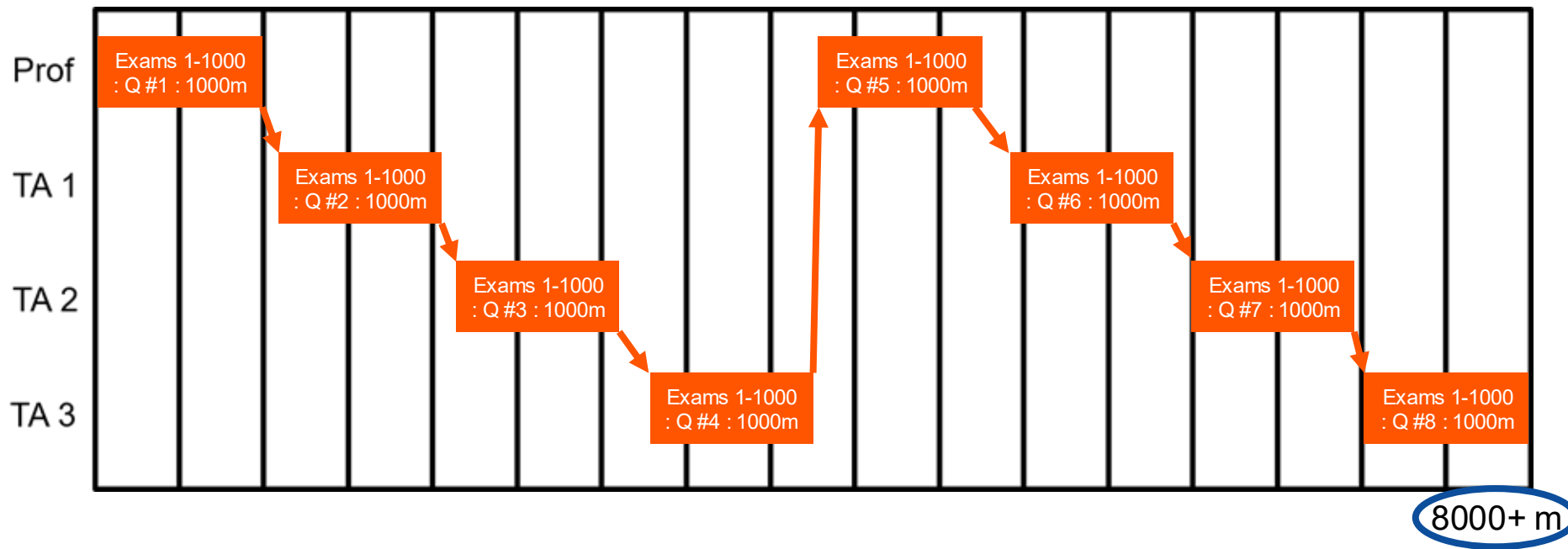


1 minute per question

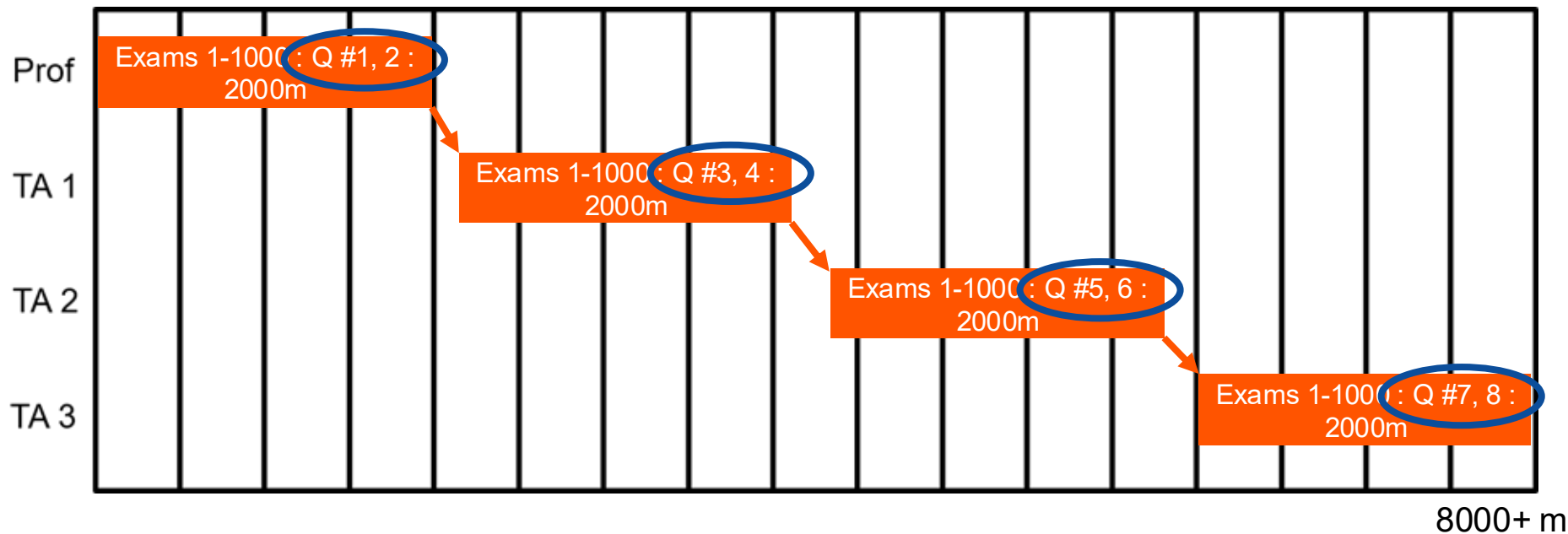
SEQUENTIAL SOLUTION



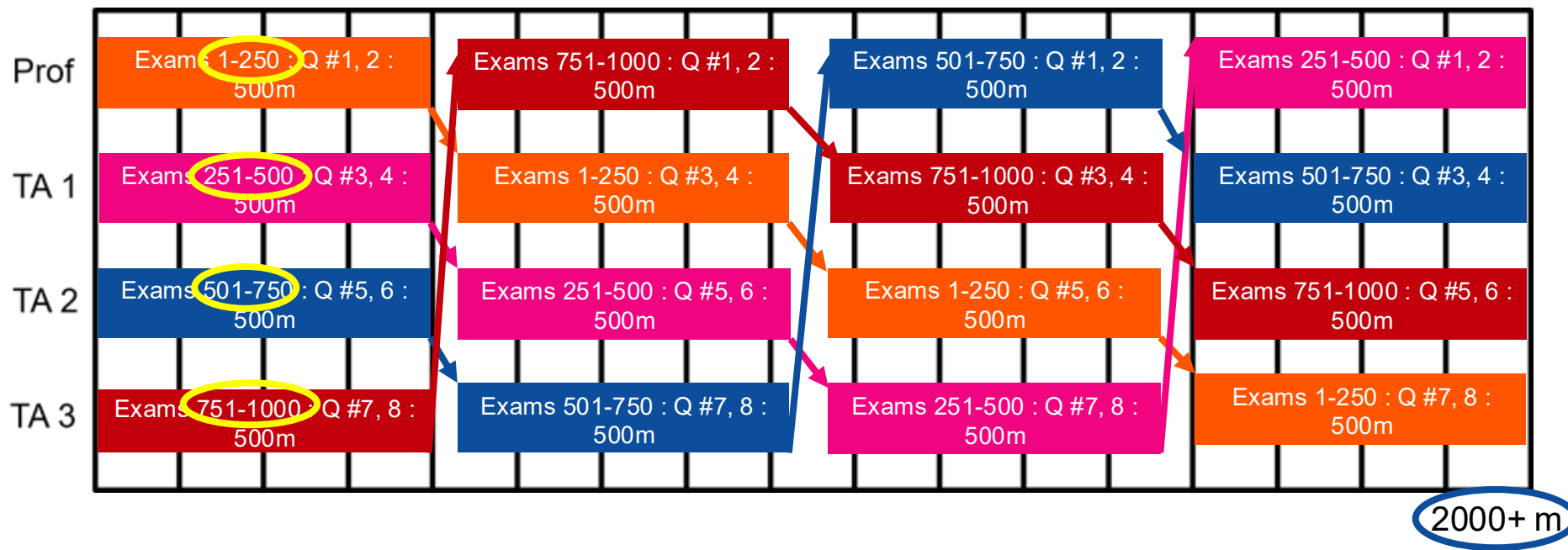
SEQUENTIAL SOLUTION



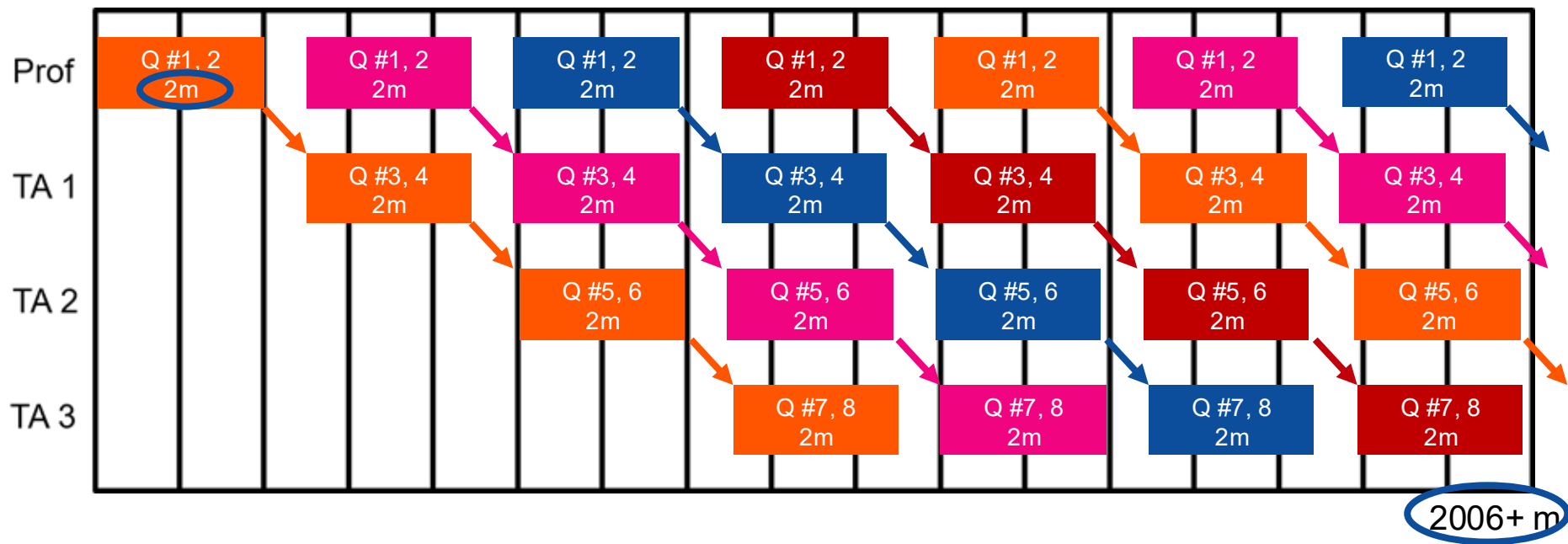
SEQUENTIAL SOLUTION



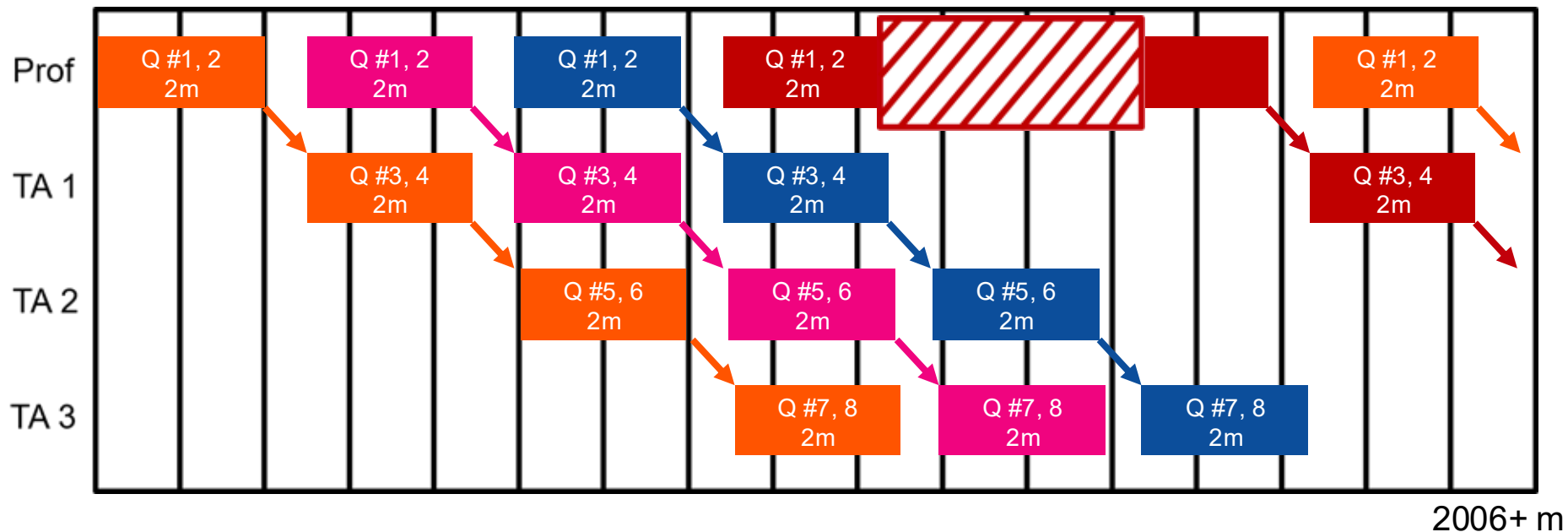
PARALLEL SOLUTION



PIPELINE



PIPELINE STALL



GRADING EXAMPLE SUMMARY

It's critical to understand the problem before trying to parallelize it

- Can the work be done in an arbitrary order, or must it be done in sequential order?
- Does each task take the same amount of time to complete? If not, it may be necessary to *“load balance.”*

In our example, the only restriction is that a single question be graded by a single grader, so we could divide the work easily, but had to communicate periodically.

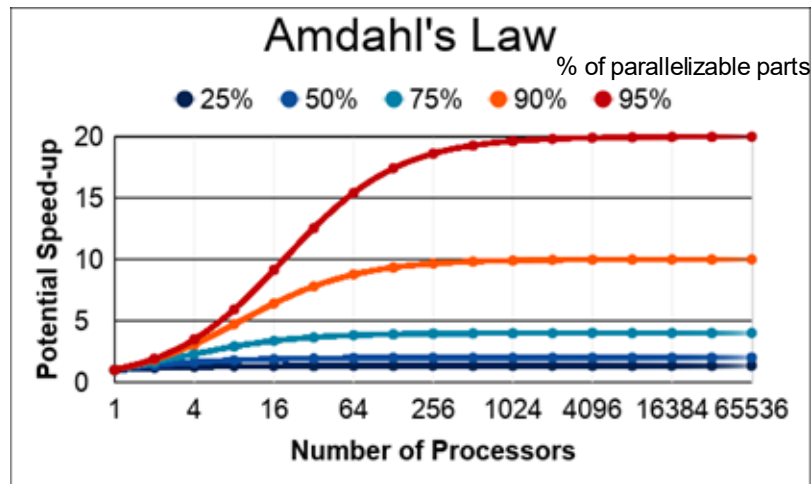
- This case study is an example of task-based parallelism. Each grader is assigned a task like “Grade questions 1 & 2 on the first 500 tests”
- If instead each question could be graded by different graders, then we could have data parallelism: all graders work on Q1 of the following tests, then Q2, etc.

AMDAHL'S LAW

AMDAHL'S LAW

Serialization Limits Performance

- Amdahl's law is an observation that how much speed-up you get from parallelizing the code is limited by the remaining serial part.
- Any remaining serial code will reduce the possible speed-up
- This is why it's important to focus on parallelizing the most time consuming parts, not just the easiest.



APPLYING AMDAHL'S LAW

Estimating Potential Speed-up

- What's the maximum speed-up that can be obtained by parallelizing 50% of the code?

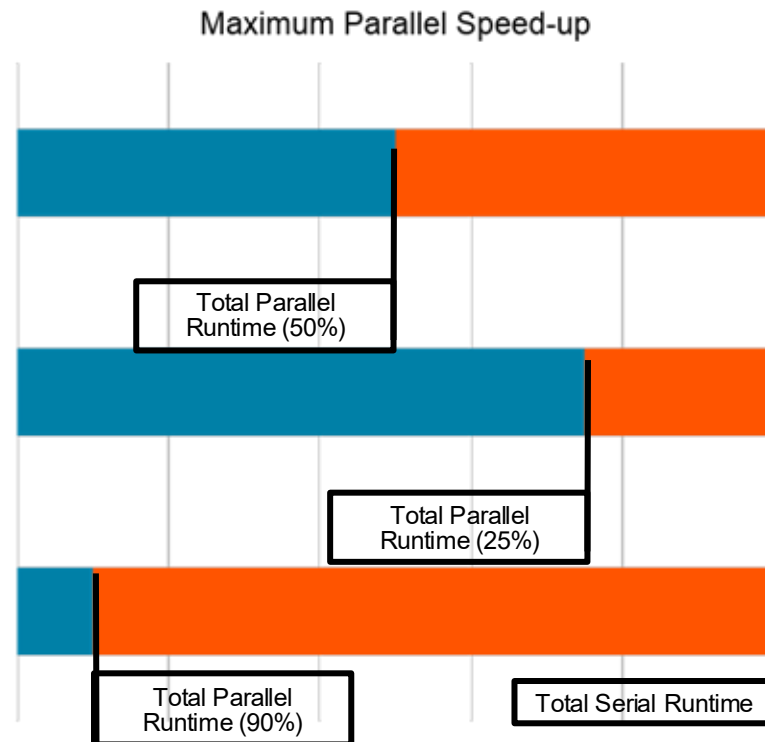
$$(1 / 100\% - 50\%) = (1 / 1.0 - 0.50) = 2.0X$$

- What's the maximum speed-up that can be obtained by parallelizing 25% of the code?

$$(1 / 100\% - 25\%) = (1 / 1.0 - 0.25) = 1.3X$$

- What's the maximum speed-up that can be obtained by parallelizing 90% of the code?

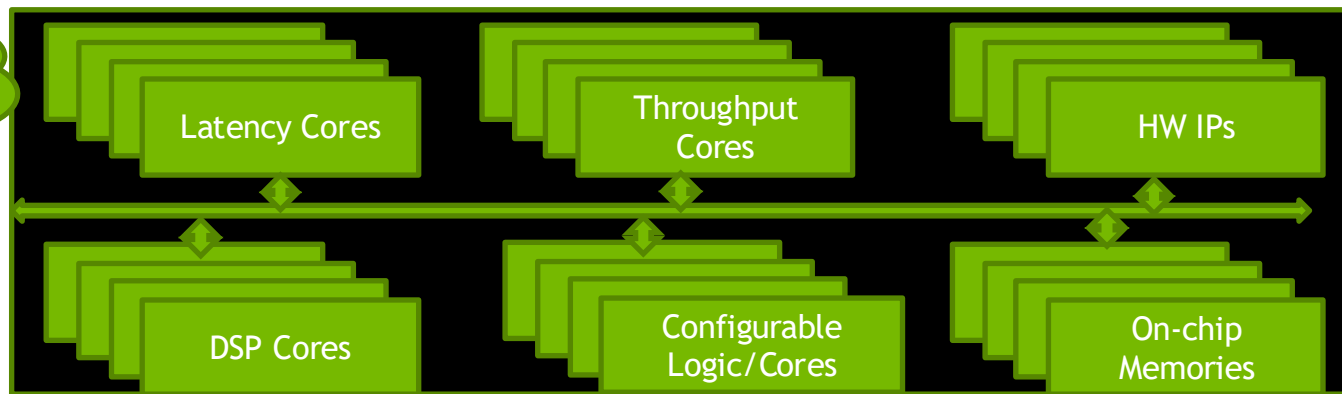
$$(1 / 100\% - 90\%) = (1 / 1.0 - 0.90) = 10.0X$$



Heterogeneous Parallel Computing

- Use the best match for the job (heterogeneity in mobile SOC)

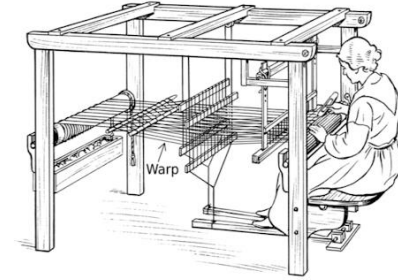
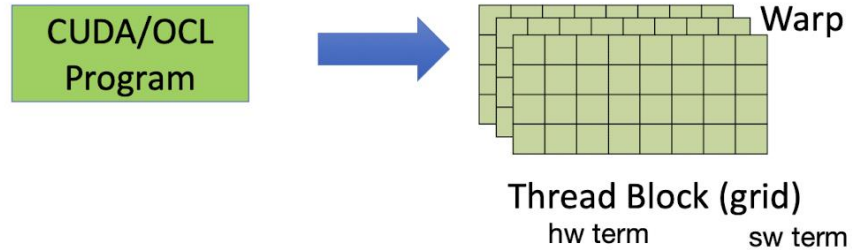
Cloud Services



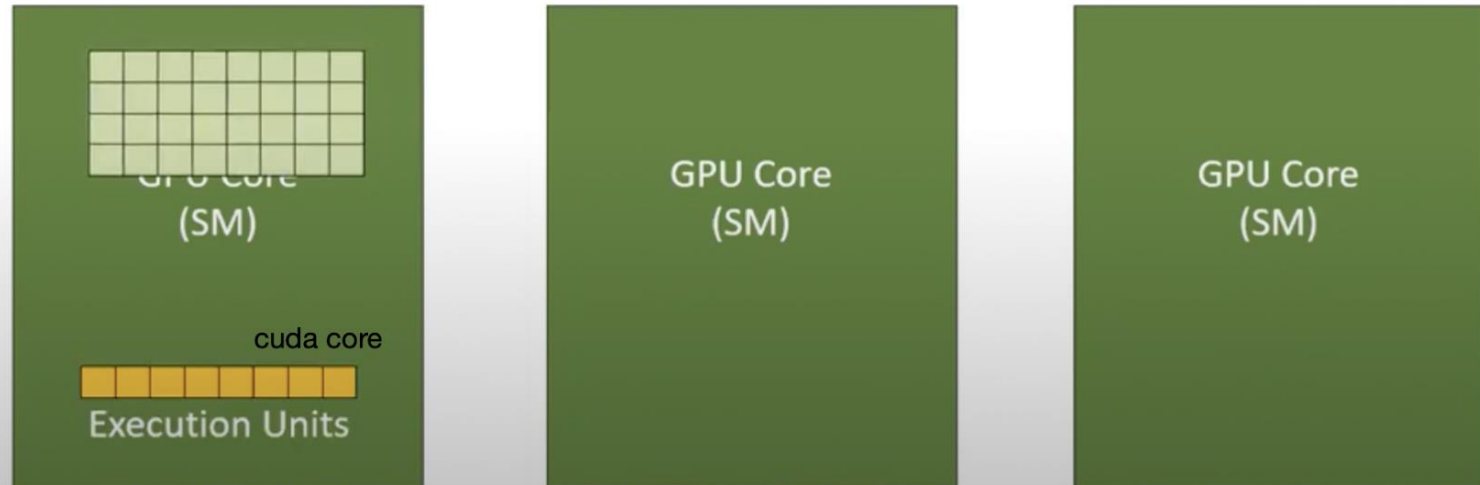
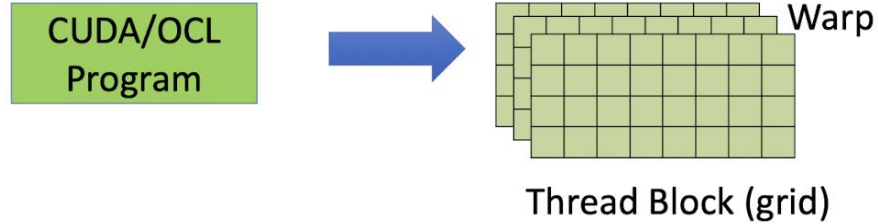
Fundamental Concepts Behind GPUs

- SIMD/SIMT
 - Needs computation hardware for highly parallelizable workloads
 - Emphasize throughput over latency
 - Need high parallelizable workloads to reduce pipeline stalls.

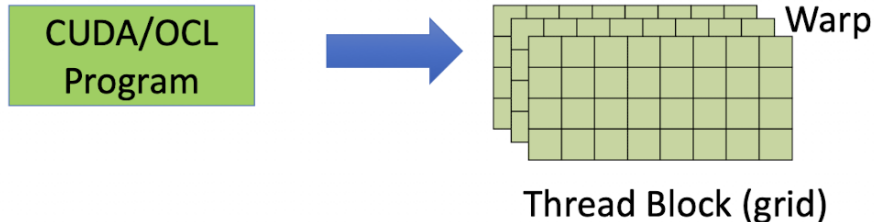
GPU Cores (SM)



GPU Cores (SM)



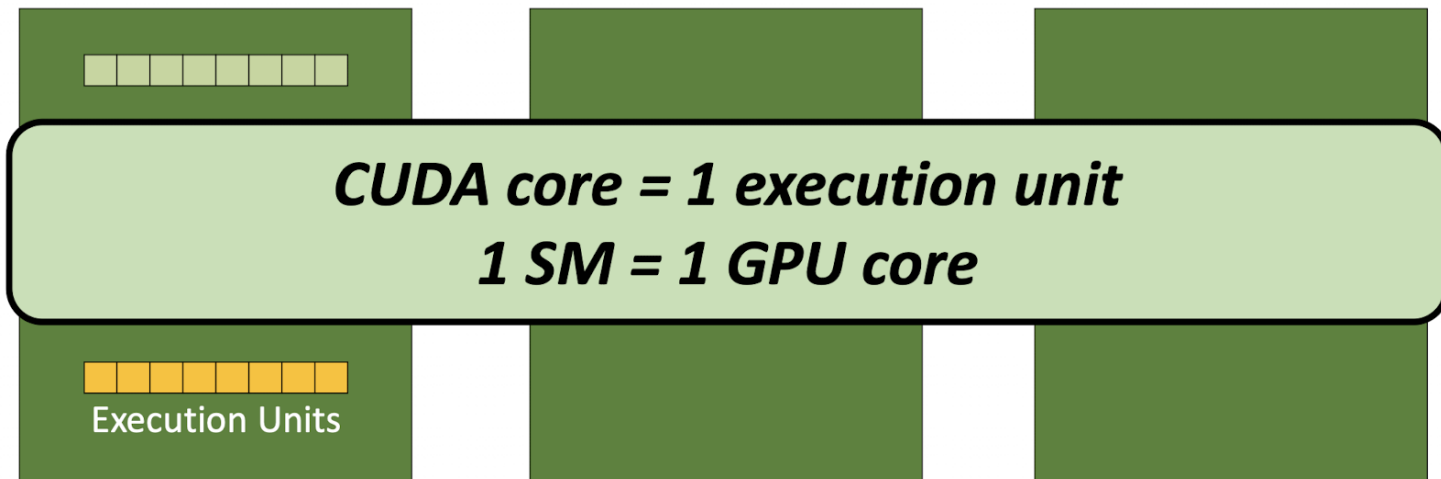
GPU Cores (SM)



Render Config	
Shading Units:	10496
TMUs:	328
ROPs:	112
SM Count:	82
Tensor Cores:	328
RT Cores:	82
L1 Cache:	128 KB (per SM)
L2 Cache:	6 MB

3090 specs

<https://www.techpowerup.com/gpu-specs/geforce-rtx-3090.c3622>

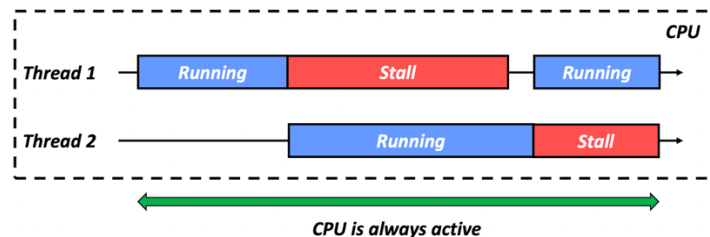


Executing a compute instruction

- 1 instruction issued at a time
 - 1 warp (NVIDIA)/wavefront (AMD) at a time
- 1 instruction issued over multiple lanes
 - Each lane has its own register files
- To reduce pipeline stalls, use **fine-grain Multithreading** to swap between threads

- Review of threads (CPU version)
- Each thread is assigned to a CPU
- If $\#threads \gg \#CPU$
 - Ex: running 10 threads in 4 CPUs
 - Each thread takes turns
 - Context switching: switches between threads
 - OS and HW create policies to context switch
- Idea: upon HW stall, switch context.

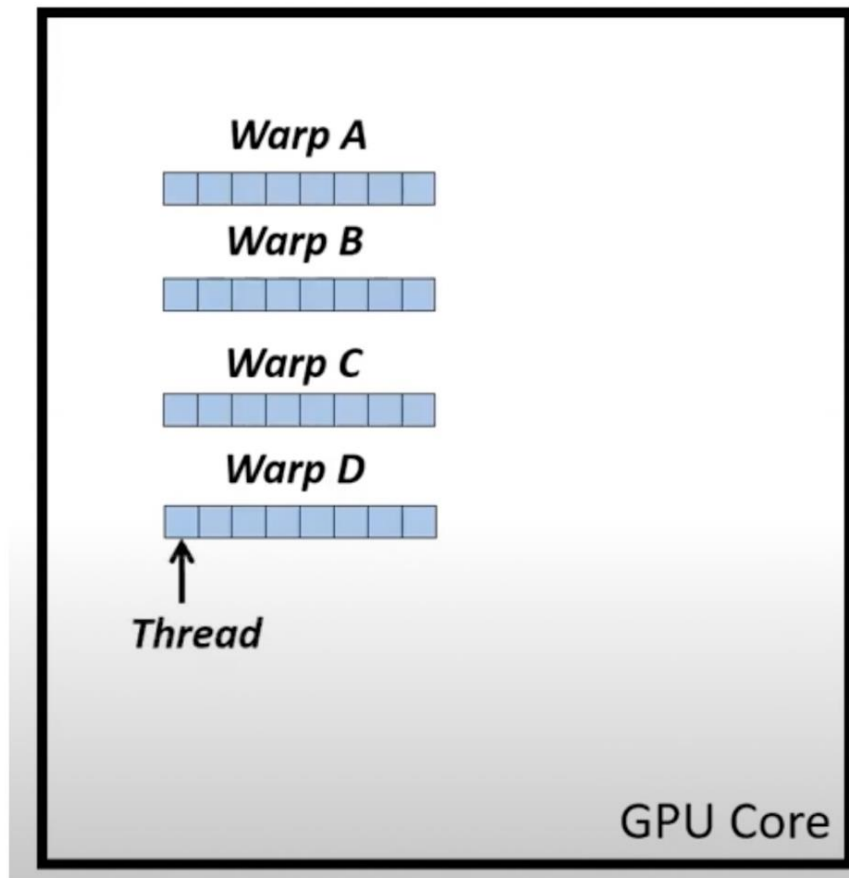
Intel's Hyperthreading runs 2 threads in 1 CPU



Fine-grain Multithreading

- GPU fine-grain Multithreading is a HW-managed multithreading scheme
- Switch to a new thread **every cycle**
- Downsides:
 - Low single thread performance
 - Extra logic and registers to keep track of threads
 - Need lots of threads to fill the pipeline

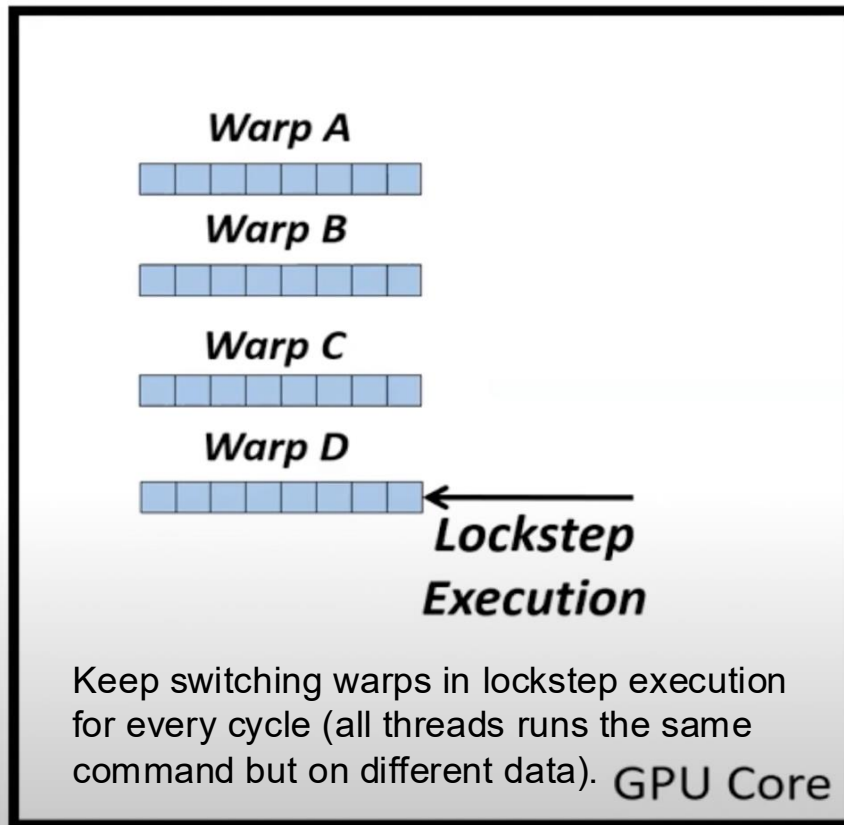
GPU Execution



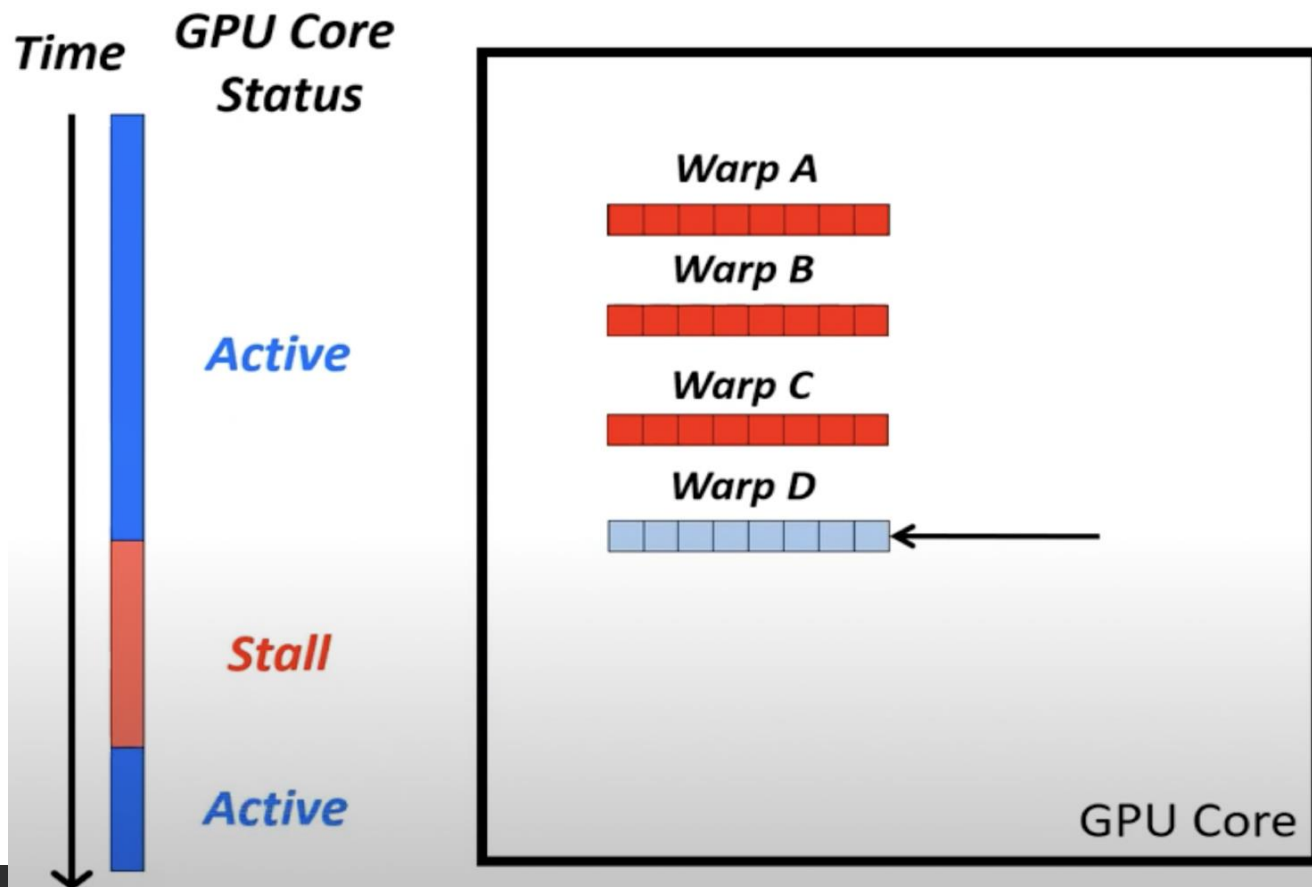
GPU Execution

Time **GPU Core
Status**

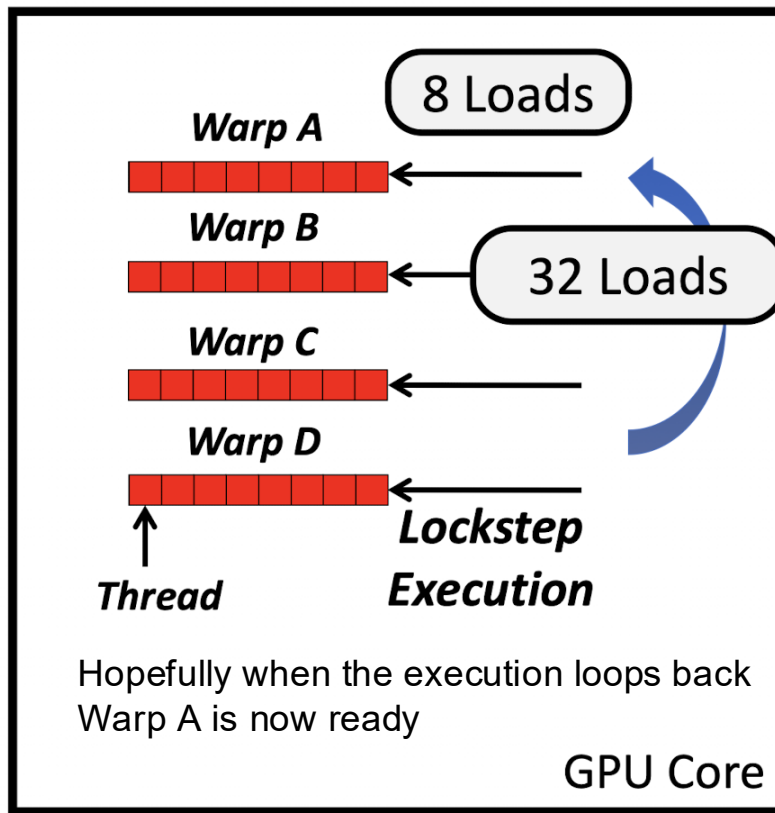
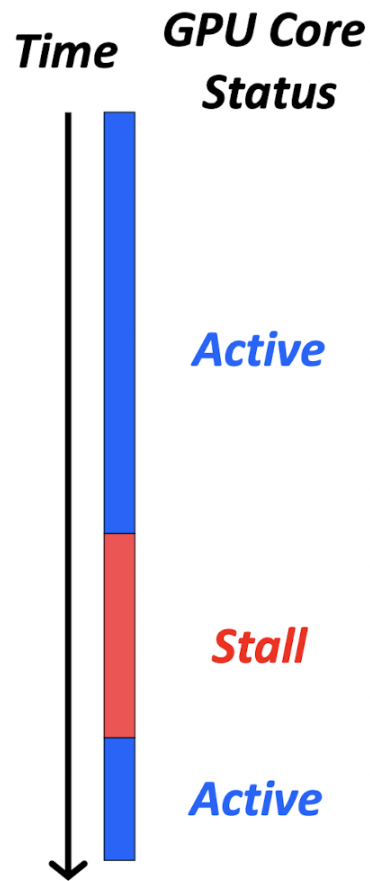
Active



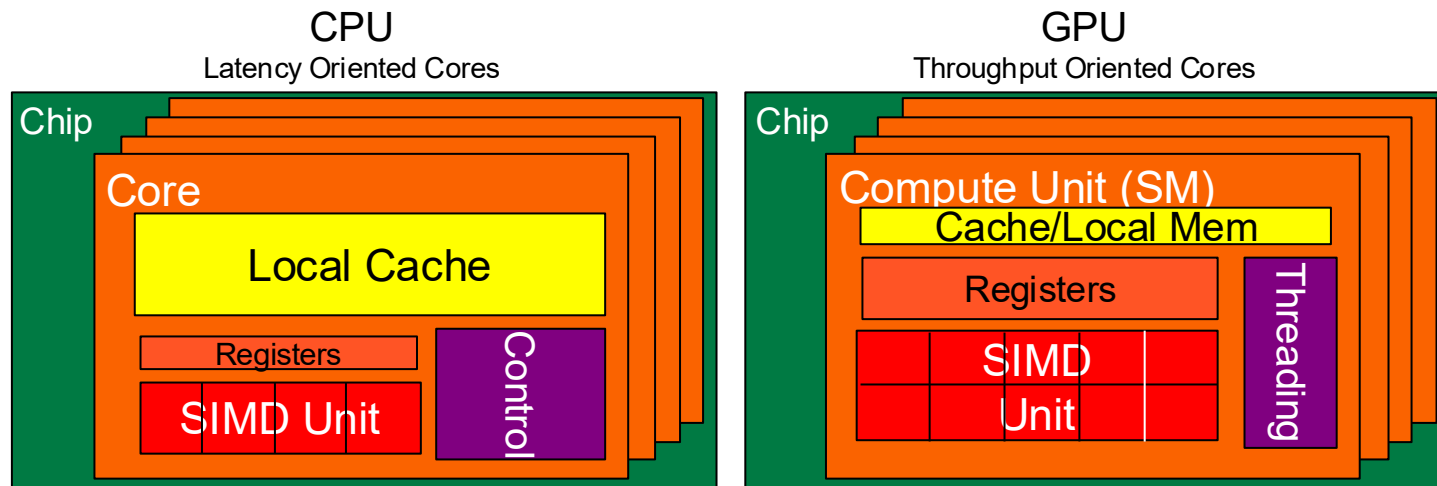
GPU Execution



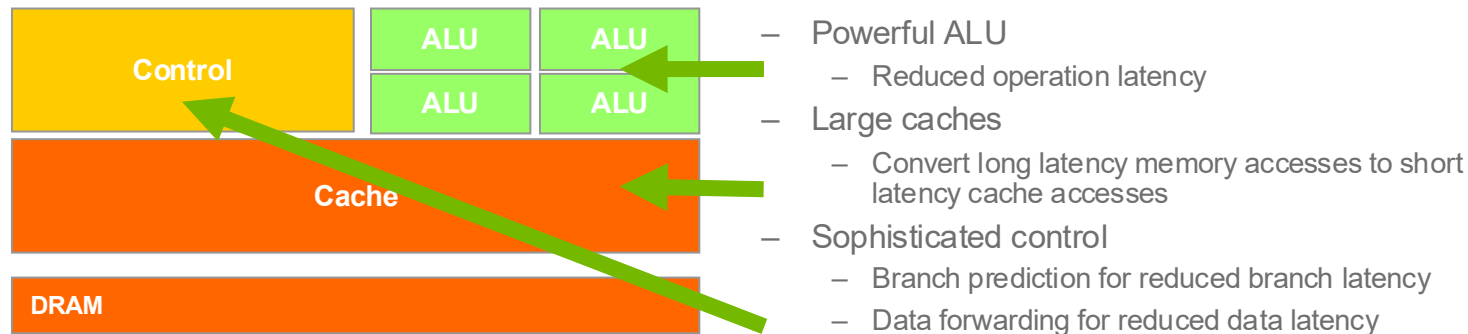
GPU Execution



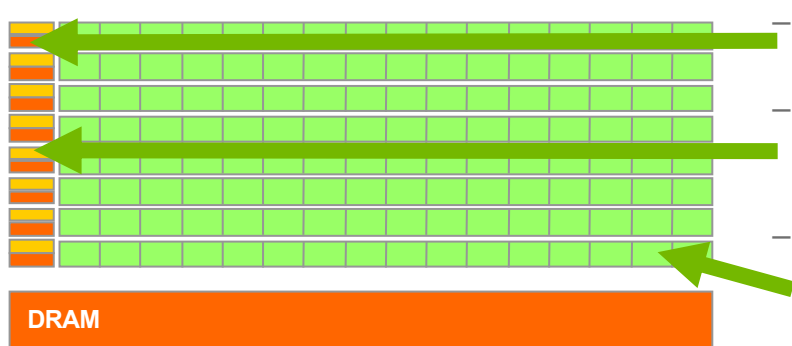
CPU and GPU are designed very differently



CPU: Latency Oriented Design



GPUs: Throughput Oriented Design



- Small caches

- To boost memory throughput

- Simple control

- No branch prediction

- No data forwarding

- Energy efficient ALUs

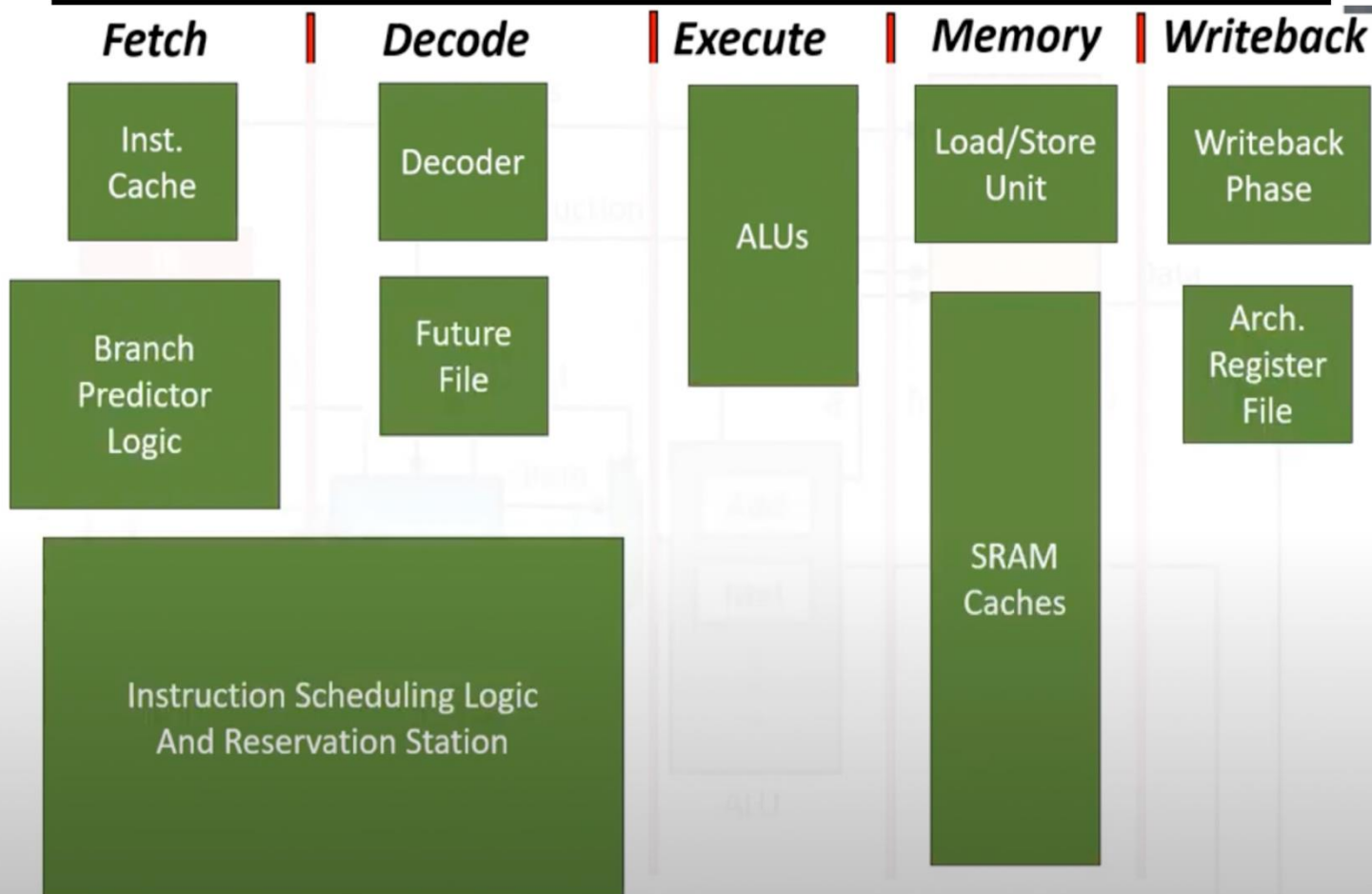
- Many, long latency but heavily pipelined for high throughput

- Require massive number of threads to tolerate latencies

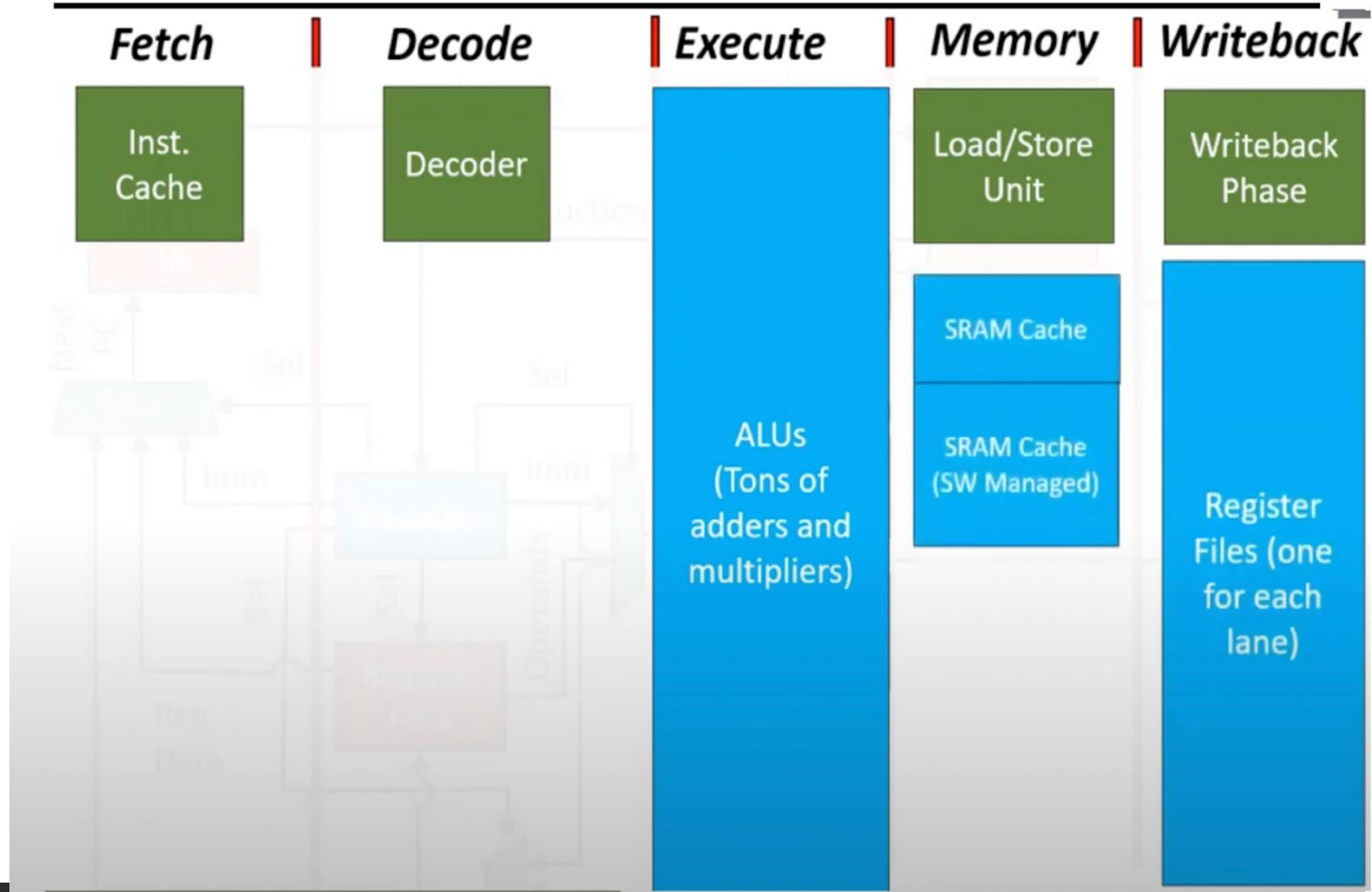
- Threading logic

- Thread state

Let's Go Back to the 5-stage Pipeline



Let's Go Back to the 5-stage Pipeline



Winning Applications Use Both CPU and GPU

- CPUs for sequential parts where latency matters
 - CPUs can be 10X+ faster than GPUs for sequential code
- GPUs for parallel parts where throughput wins
 - GPUs can be 10X+ faster than CPUs for parallel code
- If you have a serial code, GPU is very bad
- One decoder (SM) for all ALUs
 - Parallel threads must be similar
 - Otherwise it runs one thread at a time

Branch Divergence

- In an if statement, only certain threads will be active.
- But we only have one ALU
- Run all threads, but only writeback the threads that are active
- This is called Branch Divergence



```
If(a==0){  
    ...  
    do something  
    ...  
}  
else {  
    ...  
    do something else  
    ...  
}
```

Branch Divergence Example

- Consider the following code

```
for(i = 0; i < 1024; i++) {  
    if(A[i]%4 == 0){  
        A[i] = B[i] + C[i];    inst 1  
                               inst 2  
    }  
    else {  
        A[i] = D[i] - E[i];    inst 3  
        B[i] = F[i] - G[i];    inst 4  
    }  
}
```

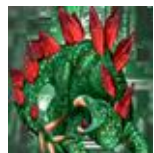
Assume the GPU has 64 lanes and load/store is magically free, what is my GPU utilization?

$$\begin{aligned}\text{GPU Util} &= \text{Actual Instructions ran} / \text{Max Instructions} \\ &= (1024 + 1024 \cdot 1/4 + 1024 \cdot 3/4 + 1024 \cdot 3/4) / 1024 \cdot 4\end{aligned}$$

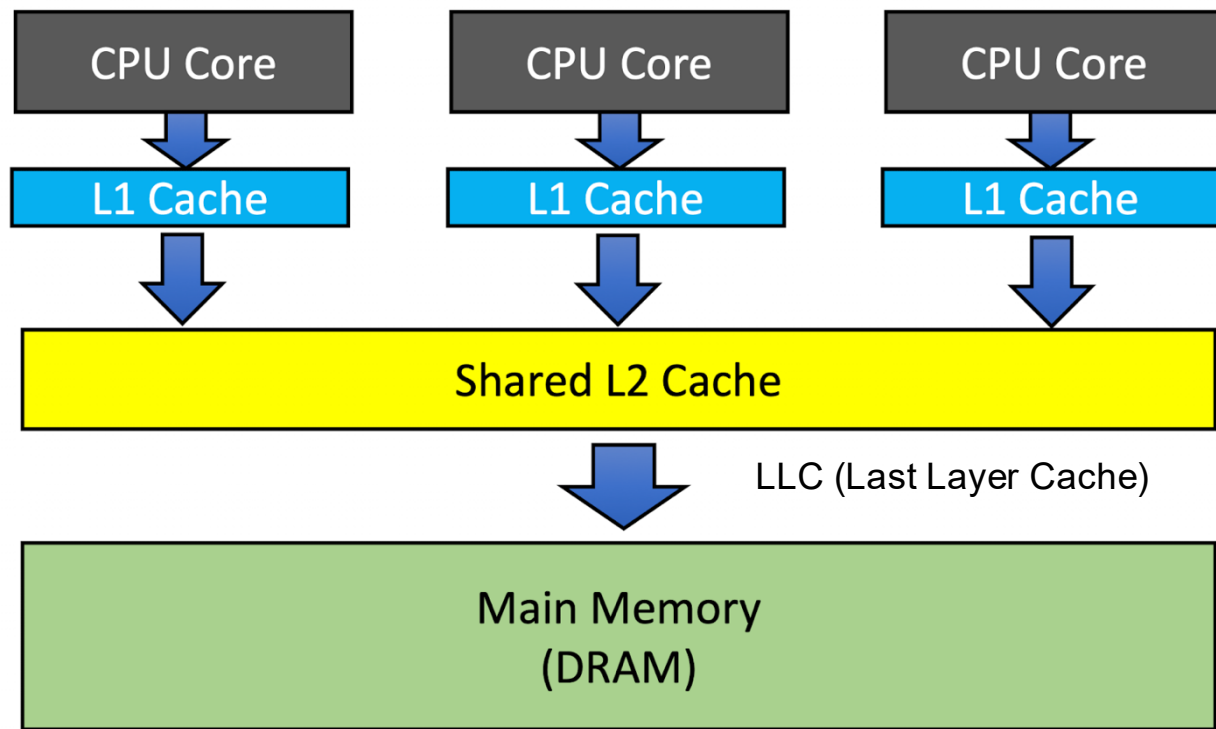


GPU and CPU memory



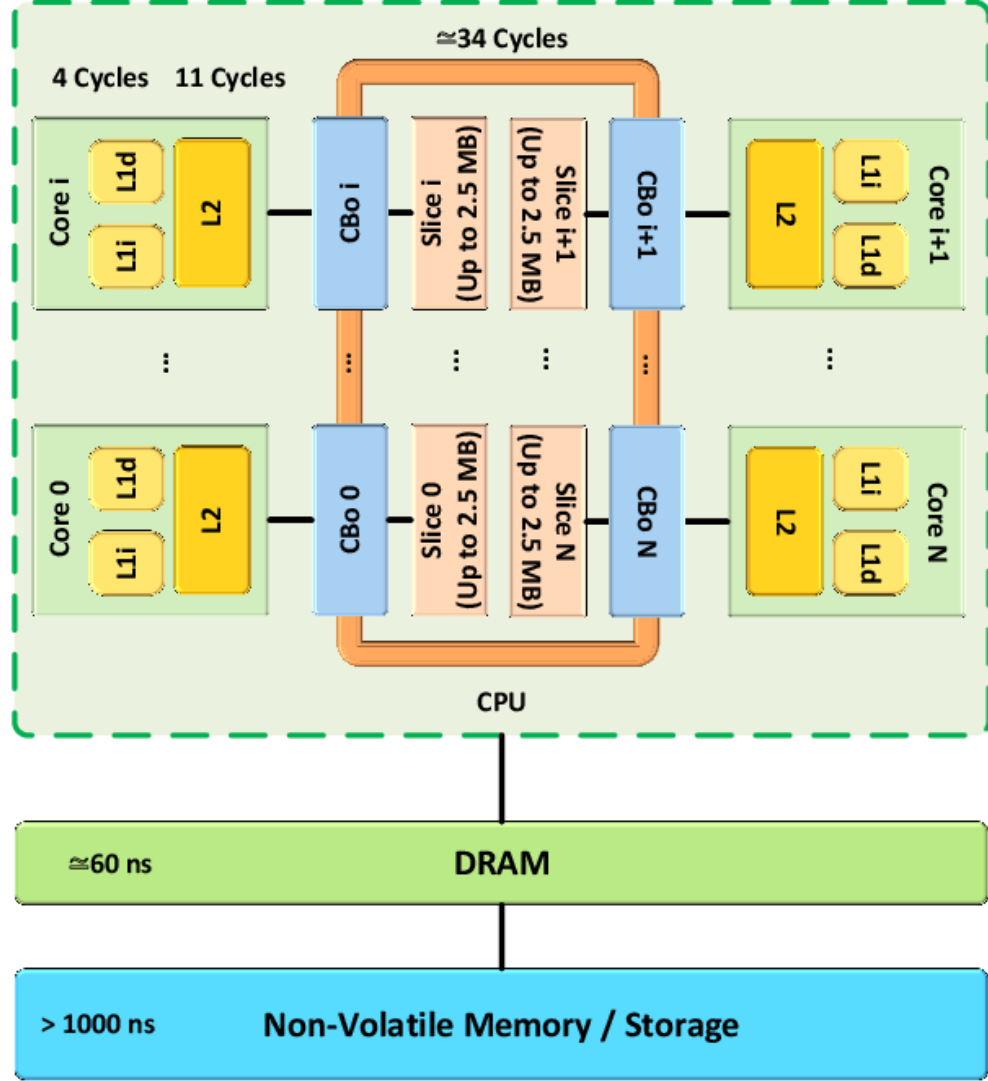


Simplified CPU Memory Hierarchy





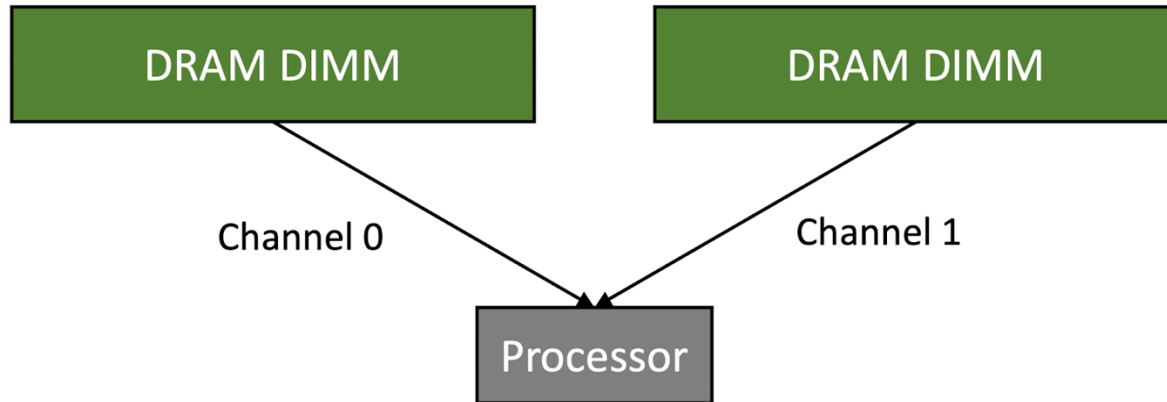
Intel Xeon Processor E5 v3
(Haswell) with N cores.



<https://www.semanticscholar.org/paper/Make-the-Most-out-of-Last-Level-Cache-in-Intel-Farshin-Roozbeh/bb76749fab841ccbe20ace74111a518f65d9870b>



Multi-channel DRAM



https://en.wikipedia.org/wiki/Multi-channel_memory_architecture



Front side → Rank 0



Back side → Rank 1



↓
DRAM chip

Difference Between Static RAM & Dynamic RAM - SRAM vs DRAM



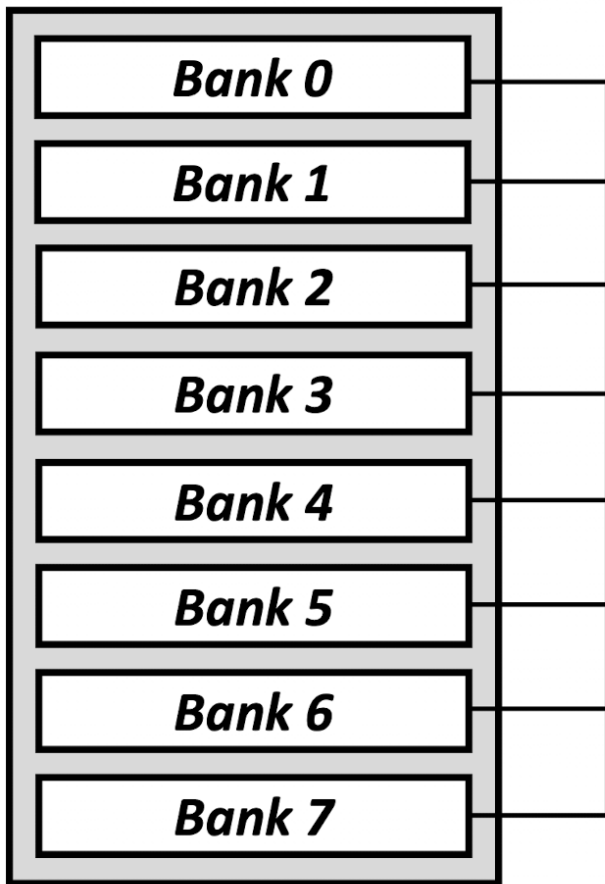
STATIC RAM (SRAM)



Dynamic Ram (DRAM)



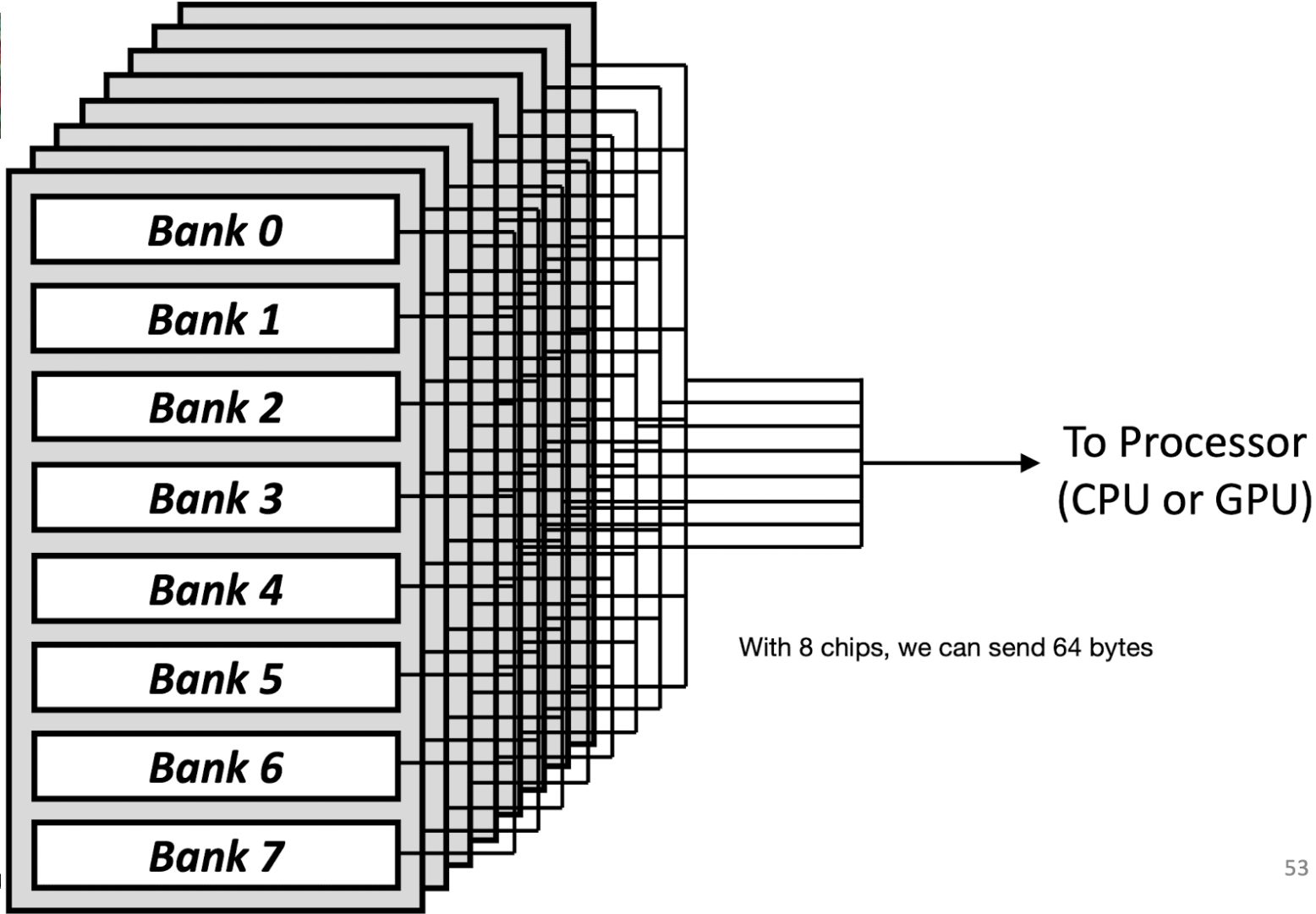
1 DRAM Chip



**Accesses to each bank
can be done
independently**

To Processor
(CPU or GPU)

Current DRAM can send
8 byte per chip



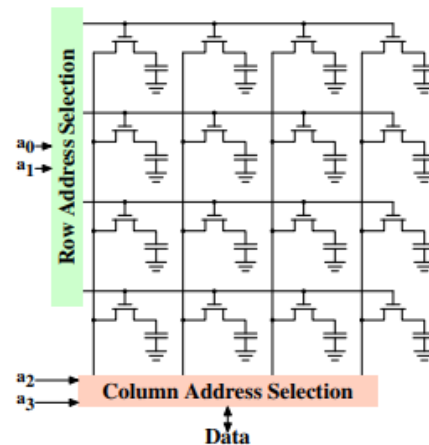
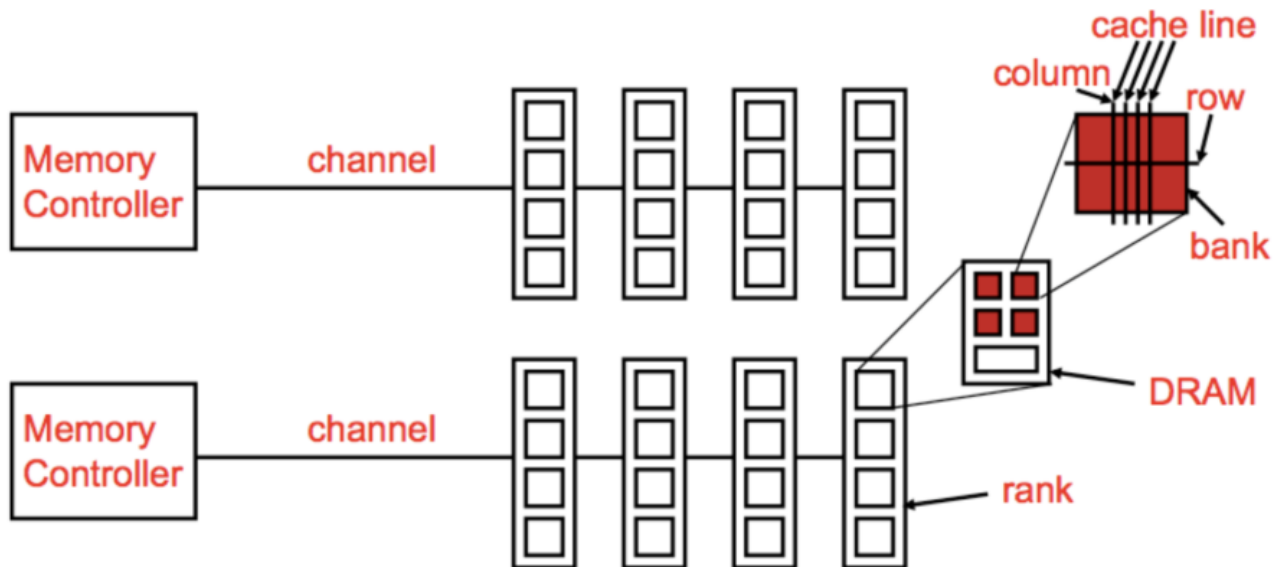
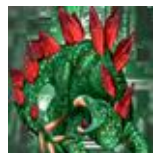


Figure 2.7: Dynamic RAM Schematic

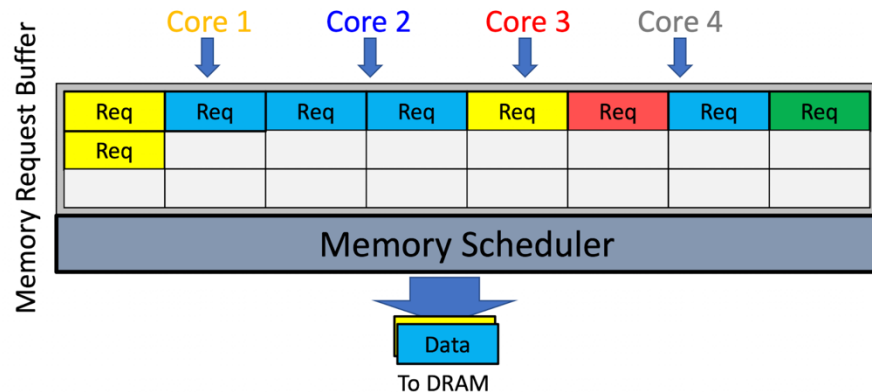
<http://thebeardsage.com/dram-nomenclature-explained/>

<https://electronics.stackexchange.com/questions/533431/why-are-dram-cells-laid-out-in-a-square-with-regards-to-demux-size>



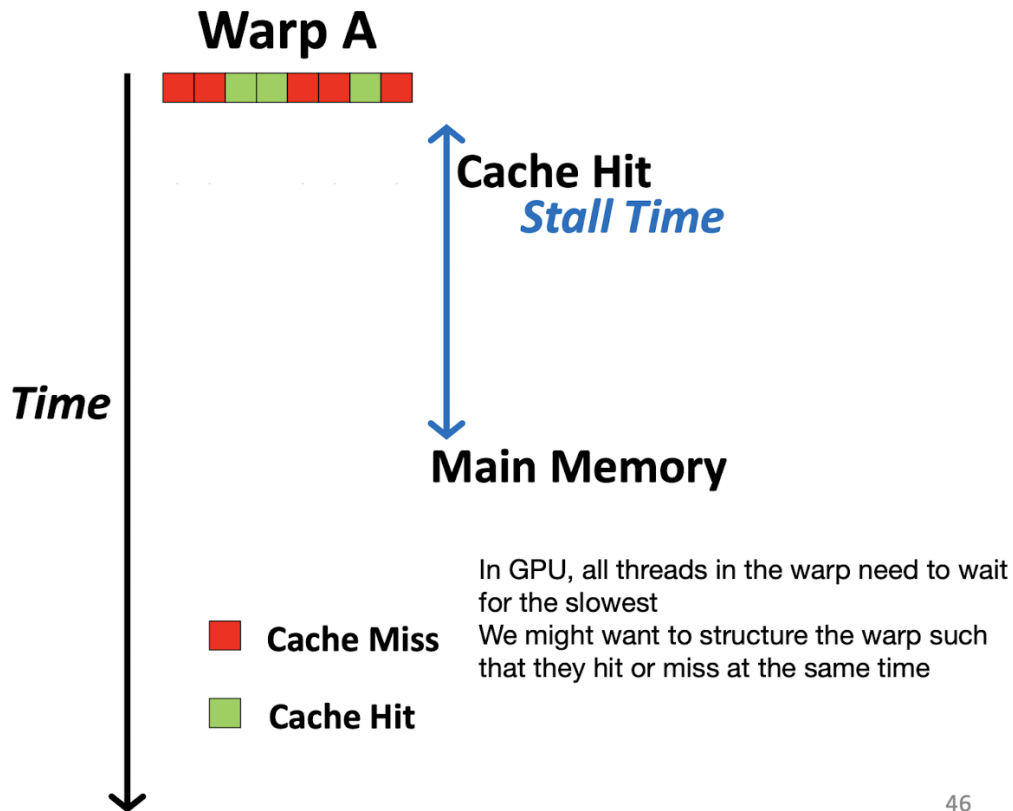
DRAM parallelism

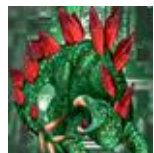
- In a DRAM array, one row can be active at a time.
 - Reading from the same row consecutively is faster than reading from a new row.
 - Clever memory read management can improve latency from good row management and parallelism from good chip reads.
 - One common policy
 - First-row-hit, first come, first serve





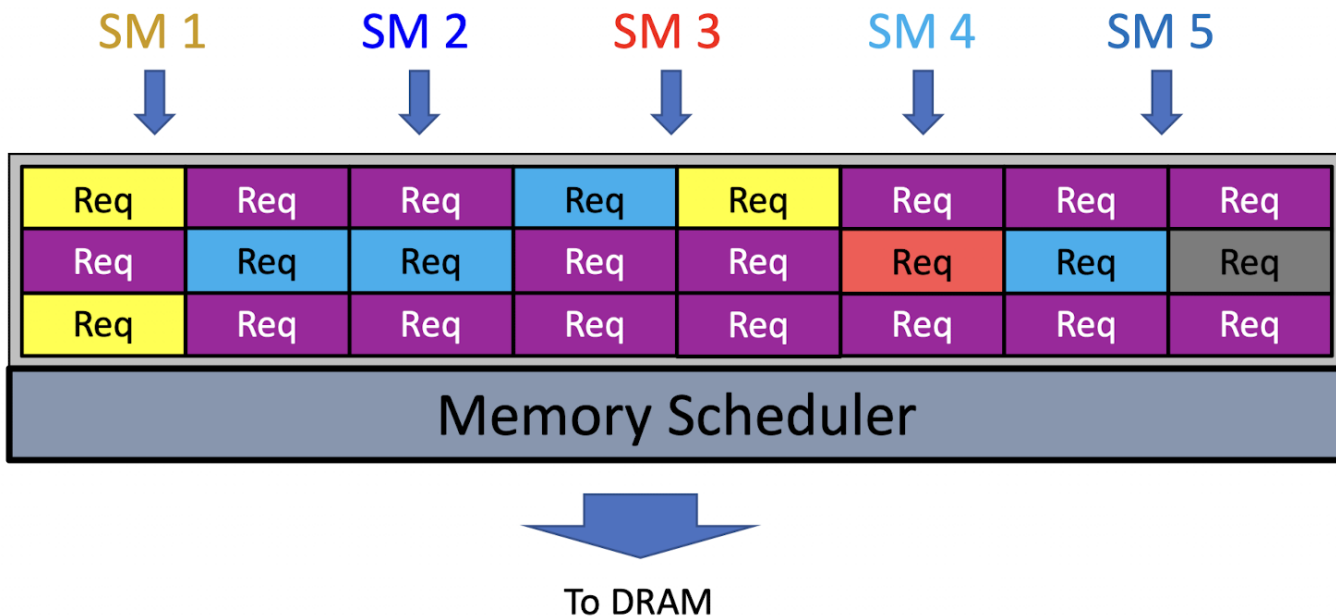
Accessing a Cache in GPUs



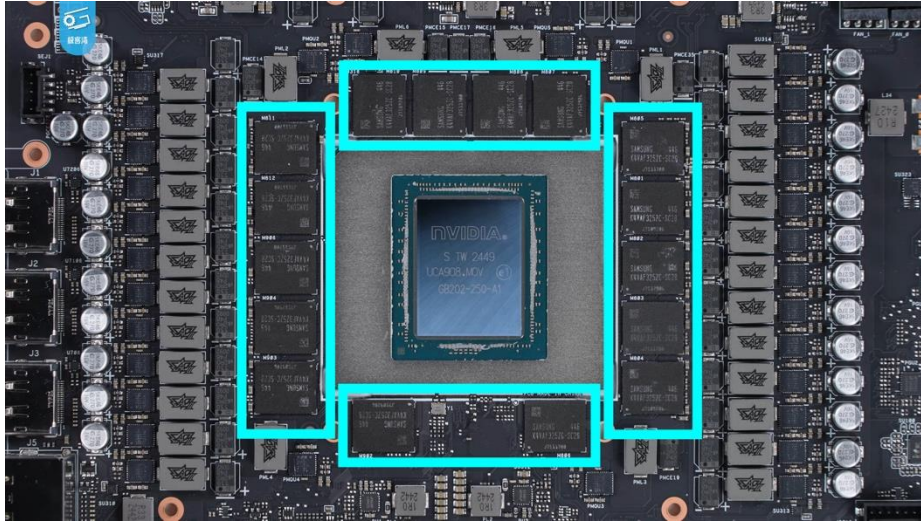


GPU memory management

- Need higher bandwidth (more cores)

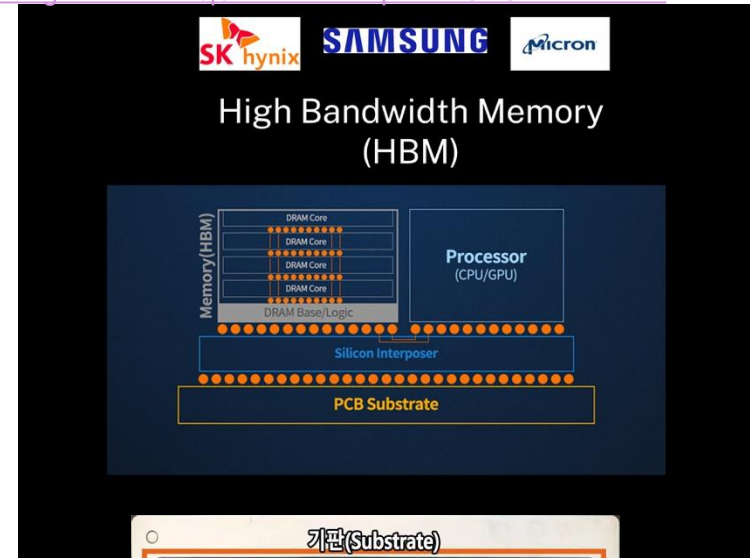


GPU DRAM



GDDR7 in 5090

https://en.wikipedia.org/wiki/GDDR7_SDRAM



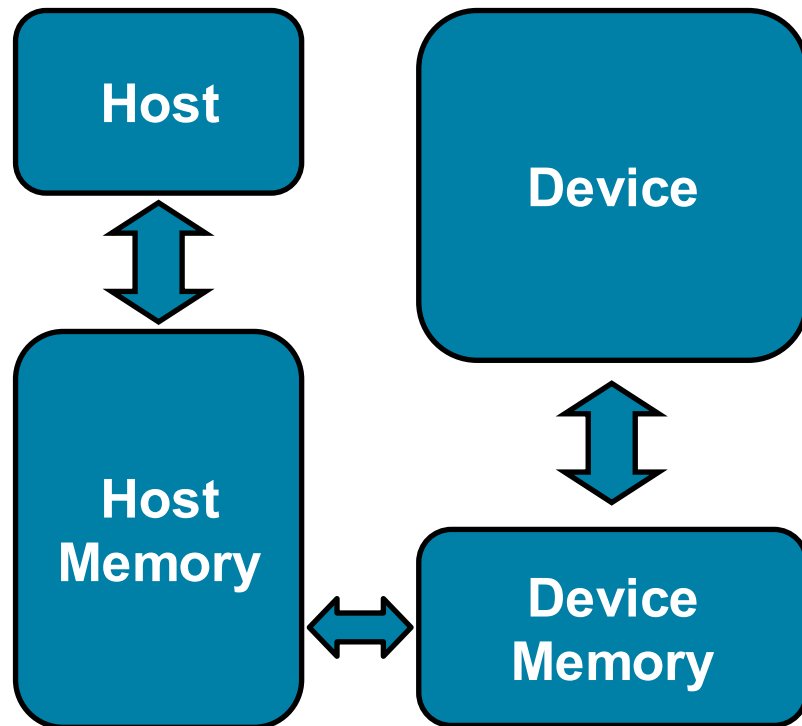
<https://en.namu.wiki/w/HBM>

EXPLICIT MEMORY MANAGEMENT

EXPLICIT MEMORY MANAGEMENT

Requirements

- Data must be visible on the **device** when we run our **parallel** code
- Data must be visible on the **host** when we run our **sequential** code
- When the host and device don't share memory, data movement must occur
- To maximize performance, the programmer should avoid all unnecessary data transfers



CPU structure

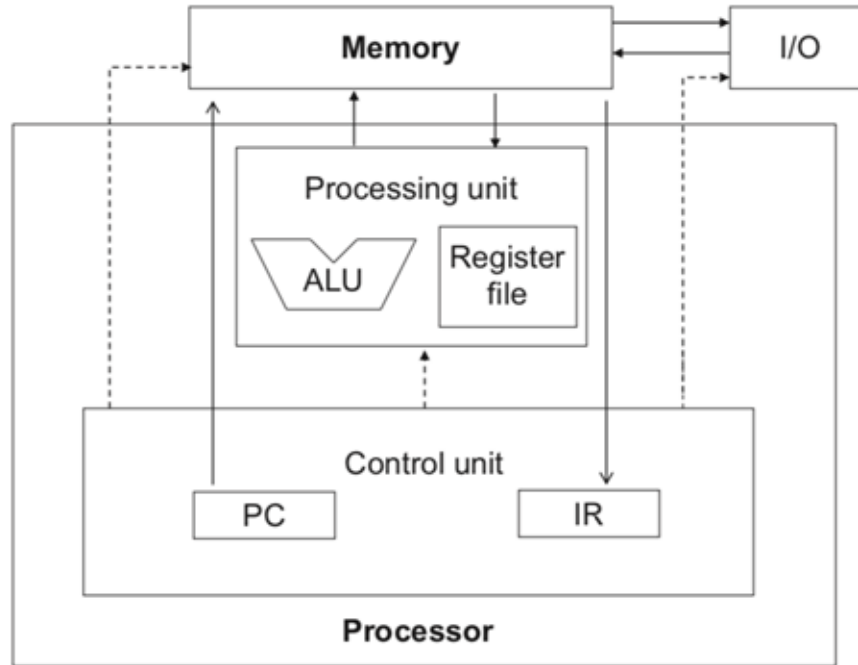


FIGURE 4.7

Memory vs. registers in a modern computer based on the von Neumann model.



CUDA GPU structure

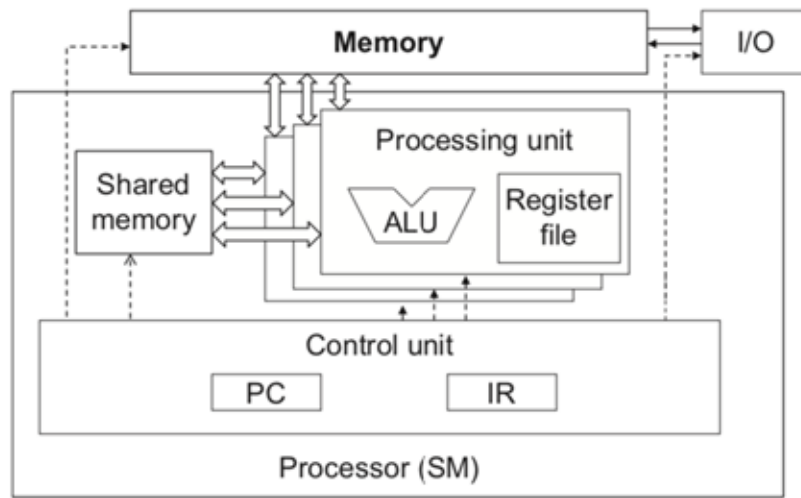


FIGURE 4.8

Shared memory vs. registers in a CUDA device SM.

A CUDA kernel is executed by a grid (array) of threads
– All threads in a grid run the same kernel code (SIMD)



CUDA memory structure

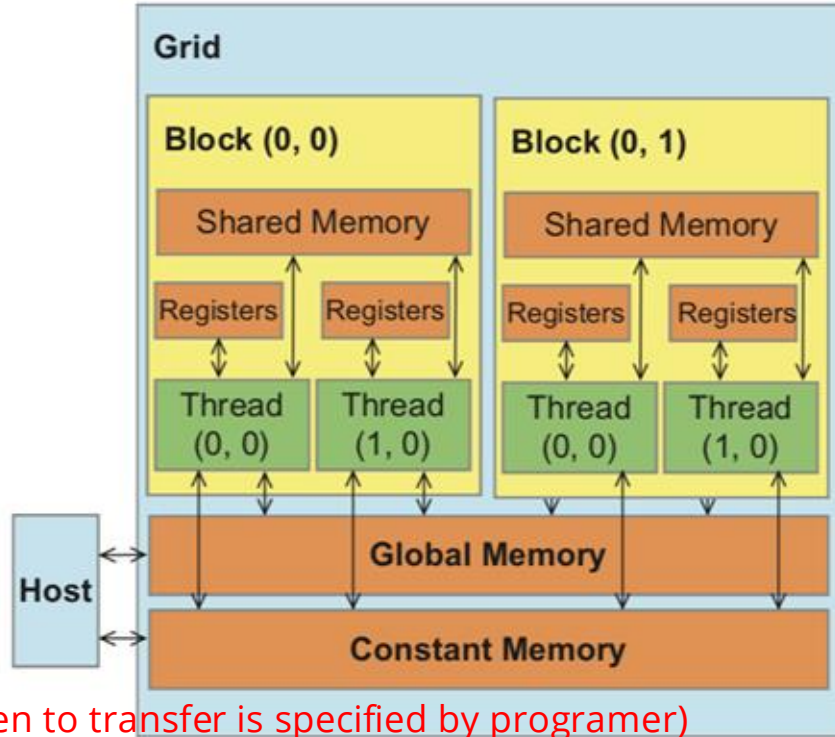
Cuda device is divided into several blocks for better memory locality.

Device code can:

- R/W per-thread registers
- R/W per-thread local memory
- R/W per-block shared memory
- R/W per-grid global memory
- Read only per-grid constant memory

Host code can

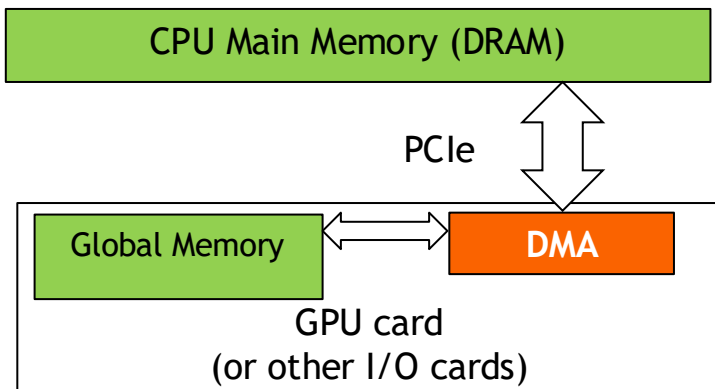
- Transfer data to/from per grid global and constant memories



Memory management (where to put, when to transfer is specified by programmer)

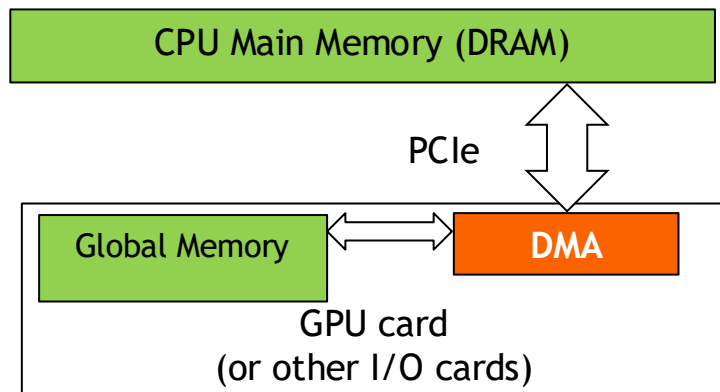
CPU-GPU Data Transfer using DMA (`cudaMemcpy()`)

- DMA (Direct Memory Access) hardware is used by `cudaMemcpy()` for better efficiency
 - Frees CPU for other tasks
 - Hardware unit specialized to transfer a number of bytes requested by OS
 - Between physical memory address space regions (some can be mapped I/O memory locations)
 - Uses system interconnect, typically PCIe in today's systems



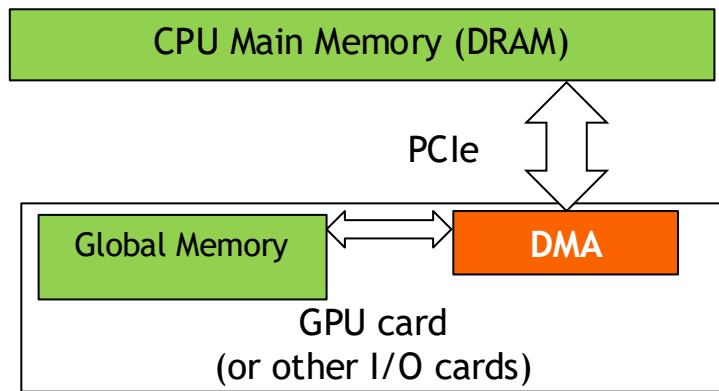
Virtual Memory Management

- Modern computers use virtual memory management
 - Many virtual memory spaces mapped into a single physical memory
 - Virtual addresses (pointer values) are translated into physical addresses
- Not all variables and data structures are always in the physical memory
 - Each virtual address space is divided into pages that are mapped into and out of the physical memory
 - Virtual memory pages can be mapped out of the physical memory (page-out) to make room
 - Whether or not a variable is in the physical memory is checked at address translation time



Data Transfer and Virtual Memory

- DMA uses physical addresses
 - When `cudaMemcpy()` copies an array, it is implemented as one or more DMA transfers
 - Address is translated and page presence checked for the entire source and destination regions at the beginning of each DMA transfer
 - No address translation for the rest of the same DMA transfer so that high efficiency can be achieved
- The OS could accidentally page-out the data that is being read or written by a DMA and page-in another virtual page into the same physical location (Solution: Pinned memory)

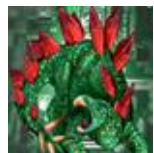


Simple Multi-Stream Host Code

```
cudaStream_t stream0, stream1;  
cudaStreamCreate(&stream0);    // stream is a FIFO queue  
cudaStreamCreate(&stream1);    // each stream can run in parallel  
  
float *d_A0, *d_B0, *d_C0; // device memory for stream 0  
float *d_A1, *d_B1, *d_C1; // device memory for stream 1  
  
// cudaMalloc() calls for d_A0, d_B0, d_C0, d_A1, d_B1, d_C1 go here
```

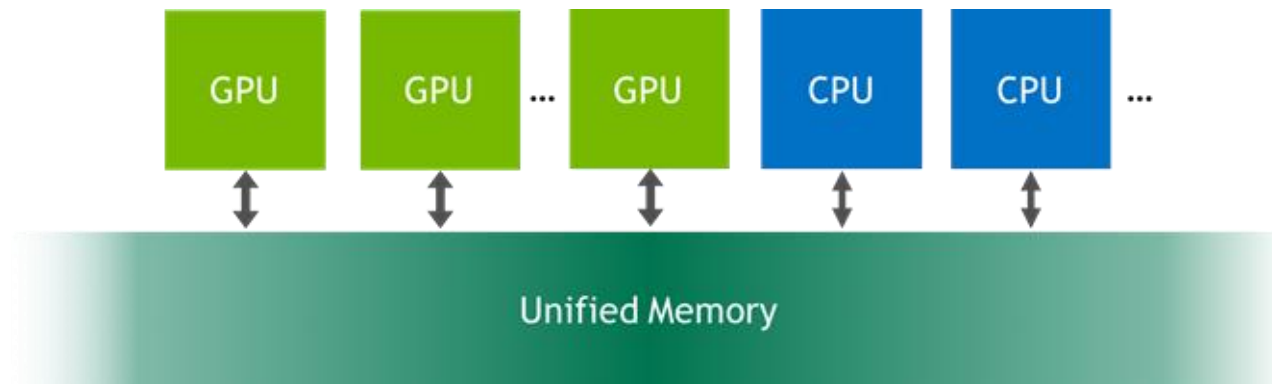
Simple Multi-Stream Host Code (Cont.)

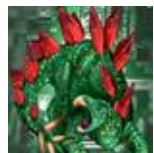
```
for (int i=0; i<n; i+=SegSize*2) {
    cudaMemcpyAsync(d_A0, h_A+i, SegSize*sizeof(float),..., stream0);    // Copy A0 in
    cudaMemcpyAsync(d_B0, h_B+i, SegSize*sizeof(float),..., stream0);    // Copy B0 in
    vecAdd<<<SegSize/256, 256, 0, stream0>>>(d_A0, d_B0,...);            // C0 = A0+B0
    cudaMemcpyAsync(h_C+i, d_C0, SegSize*sizeof(float),..., stream0);    // Copy C0 out
    cudaMemcpyAsync(d_A1, h_A+i+SegSize, SegSize*sizeof(float),..., stream1); // Copy A1 in
    cudaMemcpyAsync(d_B1, h_B+i+SegSize, SegSize*sizeof(float),...,      stream1); // Copy B1 in
    vecAdd<<<SegSize/256, 256, 0, stream1>>>(d_A1, d_B1, ...);            // C1 = A1+B1
    cudaMemcpyAsync(d_C1, h_C+i+SegSize, SegSize*sizeof(float),...,      stream1); // Copy C1 in
}
```



Unified memory

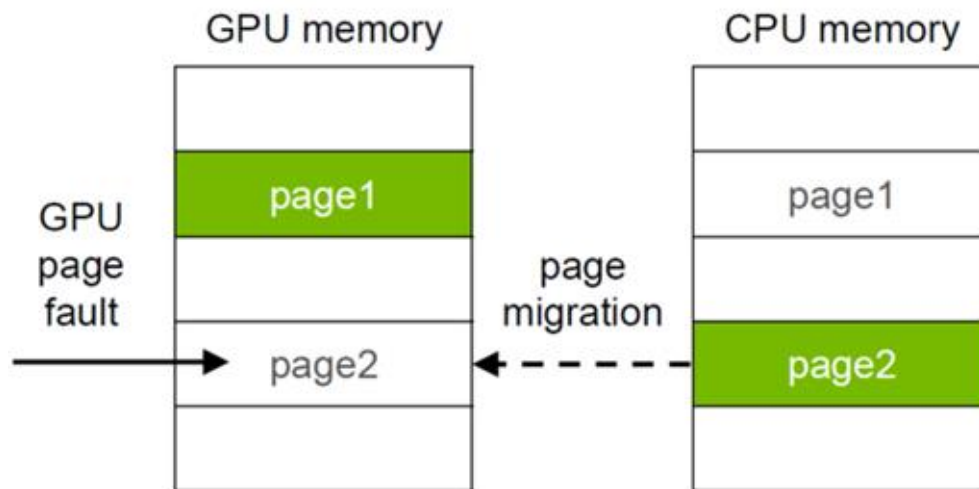
- ★ Many real-world applications do not use GPU due to its limited memory capacity, and it is the programmer's responsibility to move only necessary parts of the working set to GPU memory.
- ★ Unified Memory is a single memory address space accessible from any processor in a system.. This hardware/software technology allows applications to allocate data that can be read or written from code running on either CPUs or GPUs





Works like... Virtual memory

On-demand page migration



<https://devblogs.nvidia.com/maximizing-unified-memory-performance-cuda/>



The limit of physics

We learned about multiple trade-offs

Most of which bound by physics (Memory and bandwidth)

And other trade-offs (see HW3)

Can we do more with less memory accesses?

Complex instructions, VLIW

Downside?

Hard for compiler to optimize



Application-Specific Integrated Circuit (ASIC)

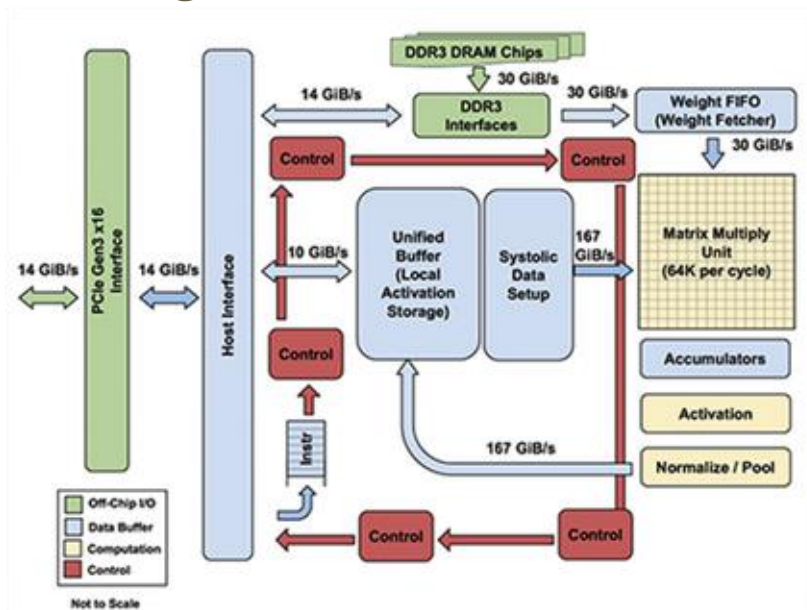
Having specific application domains

- makes compiling easier (with Application-specific language)
- smaller hardware
- less power



Google Tensor Processing Unit (TPU)

CISC instruction set that does mostly machine learning applications tasks (google search, speech-to-text, image classification)

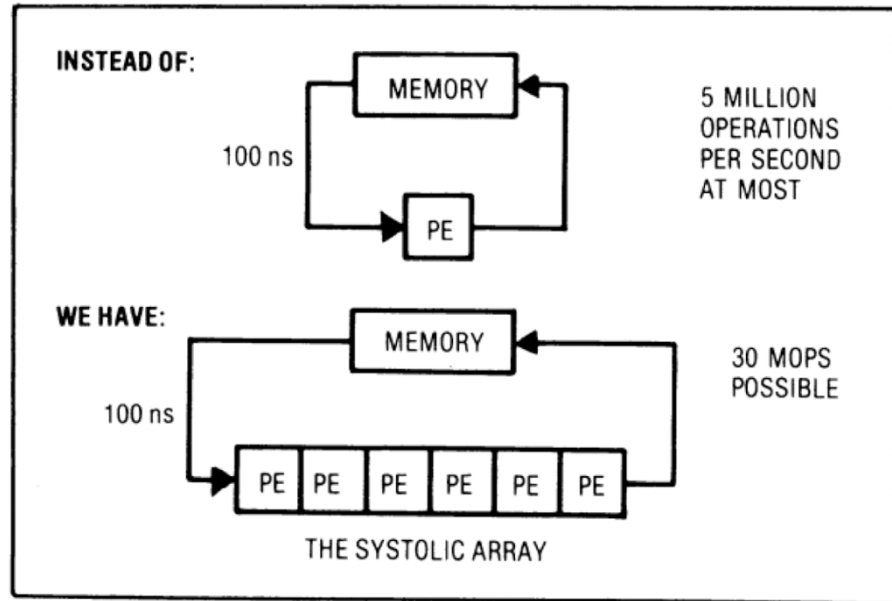


<https://cloud.google.com/blog/products/gcp/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu>



Systolic Array

H.T. Kung, "Why Systolic Architecture?", IEEE Computer 1982





Tensor core

Hardware to do multiply and add.
Specialized for matrix processing (image and machine learning applications)

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 or FP32

<https://devblogs.nvidia.com/programming-tensor-cores-cuda-9/>

	GeForce RTX 5090	GeForce RTX 5080
GPU Engine Specs:		
NVIDIA CUDA® Cores	21760	10752
Shader Cores	Blackwell	Blackwell
Tensor Cores (AI)	5th Generation 3352 AI TOPS	5th Generation 1801 AI TOPS
Ray Tracing Cores	4th Generation 318 TFLOPS	4th Generation 171 TFLOPS
Boost Clock (GHz)	2.41	2.62
Base Clock (GHz)	2.01	2.30
Memory Specs:		
Standard Memory Config	32 GB GDDR7	16 GB GDDR7
Memory Interface Width	512-bit	256-bit
Memory Bandwidth	1792 GB/sec	960 GB/sec

<https://www.nvidia.com/en-us/geforce/graphics-cards/compare/#50-series>



Logarithmic Number System hardware

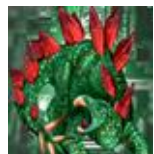
IEEE floating point format - a real number is represented by the sign, significand, and the exponent

$$1.2345 = 12345 * 10^{-4}$$

Logarithmic Number System (LNS) - a real number is represented by its Log value

$$\log_2(1.2345) = 0.30392 \text{ (stored as fixed point)}$$

Worse precision than IEEE floats



Calculation in LNS

Multiplying/dividing in LNS is simply addition/subtraction

$$b = \log(B), c = \log(C)$$

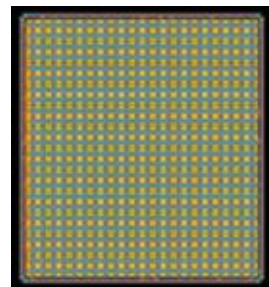
$$\log(B * C) = \log(B) + \log(C) = b + c$$

Lots of transistors saved. Smaller and faster per Watt compared to GPUs!

Addition is more expensive

$$\log(B + C) = \log(B * (1 + C/B)) = \log(B) + \log(1 + C/B) = b + G(c - b)$$

$G = \log(1 + 2^x)$ which can be computed efficiently in hardware



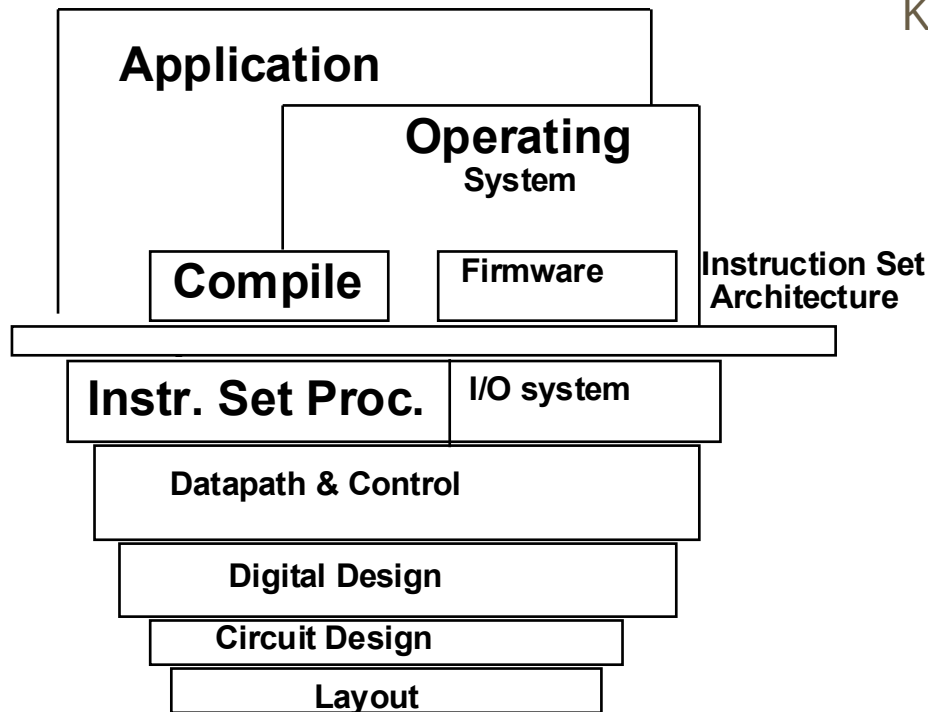
5mm

2112 cores

<https://www.technologyreview.com/s/601263/why-a-chip-thats-bad-at-math-can-help-computers-tackle-harder-problems/>

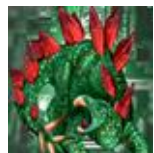


Summary & Trends



Knowing yesterday, today, and tomorrow

- CPU and GPU are being fused together
- Outsourcing of computation is becoming easier
- More application specific software and hardware (vertical integration of the system stack)
 - Collaboration between systems, hardware, and algorithms



HW 3

Yesterday, today, and tomorrow of computer architecture

CONTRIBUTED ARTICLES

A New Golden Age for Computer Architecture

By John L. Hennessy, David A. Patterson

Communications of the ACM, February 2019, Vol. 62 No. 2, Pages 48-60

10.1145/3282307

[Comments](#)

VIEW AS:



SHARE:



We began our Turing Lecture June 4, 2018¹¹ with a review of computer architecture since the 1960s. In addition to that review, here, we highlight current challenges and identify future opportunities, projecting another golden age for the field of computer architecture in the next decade, much like the 1980s when we did the research that led to our award, delivering gains in cost, energy, and security, as well as performance.

"Those who cannot remember the past are condemned to repeat it."—George Santayana, 1905

[Back to Top](#)



End of Chapter 10

